

129

49

2,979

5

6

```

graph TD
    1[1. Orkestratör (n8n)] -- "giden 443 · bağlantıyı relay kurar" --> 2[2. HR Platformu]
    2 -- "görev listesi derlet" --> 3[3. Site Relay]
    3 -- "MÜŞTERİ AĞI SİNİ" --> 4[4. İnsan]
    3 -- "yoklama" --> 5[5. Uç Nokta Ajansı]
    3 -- "Agent çalıştıramayan cihazlar" --> 6[Agent çalıştıramayan cihazlar]
    5 -- "gelen 443" --> 3
    5 -- "gelen bağlantı dinlemez" --> 3
    
```

1 · Orkestratör (n8n)
orchestrator/ – 4 dosya
devreye alma · sapma · takvim

2 · HR Platformu
platform/ – 22 dosya
katalog · tanım derleyici
şerit dağıtıcısı · kasa
Zabbix · NetBox · SIEM · ticket
39 katalog kaydı

3 · Site Relay
relay/ – 11 dosya
iş kuyruğu (Redis)
AWX yürütme düğümü
Zabbix vekili
paket önbelleği
gelen bağlantı dinlemez

4 · İnsan
human/ – 4 şablon
iş emri · yetki · kabul formu

5 · Uç Nokta Ajansı
agent/ – 8 dosya
durum bildir · tanımı çek
uygula · raporla
kendi kararı yok

Agent çalıştıramayan cihazlar
ağ anahtarı · güvenlik duvarı
hipervizör · erişim noktası

Müşteri ağına giren her bağlantı içeriden başlatılır. Relay, AWX automation mesh'te **listener_port: null** ve **peers_from_control_nodes: false** ile yapılandırılmış bir yürütme düğümüdür; gelen bağlantı dinlemez. Ajan da aynı şekilde yalnız dışarı arar. Müşterinin güvenlik duvarında hizmet sağlayıcı için tek bir gelen kural açılmaz.

Sır yönetimi: makinede koşan script'ler **.env**, Ansible adımları **ansible-vault**, n8n iş akışları ve arayüz çağrıları **n8n credentials** kullanır. Hiçbir dosyada açık metin parola yoktur; sürekli tümleştirme hattı şifresiz kasa dosyasını, depoya girmiş .env'i ve açık metin sır kalıplarını reddeder.

Dosya envanteri

Her dosyanın hangi bileşende olduğu, ne işe yaradığı ve hangi şeritlere hizmet ettiği.

DOSYA	ROL	ŞERİT	SATIR
1 · Orkestratör (n8n) — 4 dosya			
orchestrator/workflows/onboarding.json	Sözleşme imzalanınca erişim listesini üretir, formu açar, kanarya testini koşar	D	215
orchestrator/workflows/drift_reconcile.json	15 dakikada bir sapmaları uzlaştırır; etki sınırı aşılsa geri alır	E	205
orchestrator/workflows/scheduled_monthly_report.json	Ayın 1'i 06:00'da aylık raporu üretir	E	160
orchestrator/README.md	Akışların nasıl sürümlendiği ve sırların neden akışta durmadığı	E	21
2 · HR Platformu — 22 dosya			
platform/app/main.py	Giriş noktası; katalog tutarsızsa açılışta patlar	D	29
platform/app/config.py	Tüm sırlar ortam değişkeninden; kodda sabit değer yok	D	35
platform/app/models.py	Kiracı, hak seti, cihaz, sürümlü tanım, görev koşusu, denetim kaydı	D	107
platform/app/catalog/loader.py	Katalog yükleyici; 'ilk görev kasa olmalı' kuralını zorunlu kılar	D	143
platform/app/services/task_graph.py	Ön koşullara göre sıralama; envanter her şeyden önce gelir	D	52
platform/app/services/spec_compiler.py	Hak seti + cihaz rolü → arzu edilen durum; çıkarmalı çalışır	D	120
platform/app/services/dispatcher.py	Her görevi kendi şeridinin yürütücüsüne yollar — altı şeridin bulunduğu yer	A B C D E F	119
platform/app/services/agent_queue.py	A şeridi: görev relay kuyruğuna yazılır, ajan çeker	A	36
platform/app/services/drift.py	Sapma uzlaştırma ve etki sınırı koruması	A B C D	64
platform/app/security.py	Ajan kimliği ve kiracı sınırı	A	59
platform/app/integrations/awxy.py	B şeridinin yürütücüsü; relay'i dışı doğru eşleşen yürütme düğümü olarak kaydeder	B	53
platform/app/integrations/vault.py	Kasa; kiracı ayrışması yol düzeyinde zorunlu	D	58
platform/app/integrations/netbox.py	Envanter; kimlik anahtarı ajan kimliği + seri no	C D	62
platform/app/integrations/zabbix.py	İzleme; satılan şey host kaydı ve uyarı kuralıdır	D	55
platform/app/integrations/ticketing.py	F şeridinin arayüzü; kabul kriteri olmadan iş emri kapanmaz	F	58
platform/app/api/routers/agents.py	Ajan uçları; bağlantıyı her zaman ajan kurar	A	63
platform/app/api/routers/specs.py	Tanım derleme, kuru çalıştırma ve dağıtım	D	60
platform/app/catalog/services/switch-management.yaml	Örnek katalog kaydı — 39 hizmetin her biri böyle tanımlı	B	47

DOSYA	ROL	ŞERİT	SATIR
platform/tests/test_catalog.py	Tasarım kurallarını koda bağlayan testler	D	68
platform/tests/test_spec_compiler.py	Ön koşul, sökülüm ve kasa kuralının testleri	D	64
.github/workflows/ci.yml	Şifresiz kasa, depoya girmiş .env ve açık metin sır kalıbını reddeder	D	72
README.md	Beş bileşen, şerit karşılıkları ve kasa tablosu	D	60

3 · Site Relay — 11 dosya

relay/docker-compose.yml	Relay yığını; hiçbirisi internete açılmaz	A B C	57
relay/receptor/receptor.conf	Yürütme düğümü: gelen bağlantı dinlemez, kendisi dışarı bağlanır	B	34
relay/relay_api/main.py	Ajanların tek muhatabı	A	61
relay/relay_api/queue.py	İş kuyruğu; cihaz kimliğine göre bölümlenir	A	67
relay/ansible/ansible.cfg	Parolalar playbook'ta değil kasada	B	17
relay/ansible/inventory/netbox.yml	Makine listesi elle tutulmaz; envanterden gelir	B	17
relay/ansible/roles/switch_base_line/tasks/main.yml	Geri sayımlı yeniden başlatma ile korunan switch ayarı	B	37
relay/ansible/roles/firewall_base_line/tasks/main.yml	HA sanal adresine değil birincil düğüme basar	B	35
relay/ansible/playbooks/site.yml	Hangi rolün koşacağını envanterdeki cihaz rolü belirler	B	20
relay/ansible/playbooks/teardown.yml	Çıkış hattı: giriş hattının aynadaki hâli	B D	21
relay/ansible/group_vars/all/vault.yml	ansible-vault ile şifreli; şifresiz hâli CI tarafından reddedilir	B	15

4 · İnsan — 4 dosya

human/templates/access_pack.md	0. gün erişim paketi; liste paketten otomatik üretilir	F	26
human/templates/acceptance.md	Proje → yönetilen devri; sağlık yeşil olmadan kapanmaz	F	21
human/templates/pentest_authorization.md	İmzasız test başlamaz	F	20
human/runbooks/dr_failover.md	Failover'ı kimin ilan edeceği isimle yazılı	F	22

5 · Uç Nokta Ajanı — 8 dosya

agent/hragent/__main__.py	Ajan döngüsü; ilk gün ile 400. gün aynı	A	73
agent/hragent/client.py	Bağlantıyı her zaman ajan kurar: giden 443	A	50
agent/hragent/runner.py	Yalnız A şeridini yürütür; Ansible işini reddeder	A	41
agent/hragent/secrets.py	` .env ` okur; fazla açık izinli dosyayı reddeder	A	60
agent/hragent/inventory.py	Kimlik anahtarı kalıcı ajan kimliği — klon makineler için	A	79
agent/hragent/apply/packages.py	Paketler relay önbelleğinden; kurulu ise dokunmaz	A	55
agent/hragent/apply/files.py	Sır dosyaları daima 0600	A	37
agent/tests/test_runner.py	Ajanın kendi şeridi dışına çıkmadığının testi	A	29

1 · Orkestratör (n8n)

Sözleşmeden sonra ne olacağını sıraya koyan iş akışları

orchestrator/workflows/onboarding.json

Sözleşme imzalanınca erişim listesini üretir, formu açar, kanarya testini koşar

şerit: D – Platform kurulumu · 215 satır · json

```
{
  "name": "onboarding",
  "active": false,
  "settings": {
    "executionOrder": "v1"
  },
  "tags": [
    {
      "name": "onboarding"
    },
    {
      "name": "lane:D"
    }
  ],
  "nodes": [
    {
      "parameters": {
        "path": "contract-signed",
        "httpMethod": "POST",
        "options": {
          "rawBody": false
        }
      },
      "id": "sözleşme-imzalandı",
      "name": "Sözleşme imzalandı",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 1,
      "position": [
        0,
        300
      ]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "={{ $env.HR_PLATFORM_URL }}/specs/compile",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={{ JSON.stringify({tenant: $json.tenant, entitlements: $json.entitlements, dry_run: true}) }}",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth"
      },
      "id": "erişim-listesi-üret",
      "name": "Erişim listesi üret",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 1,
      "position": [
        220,
        300
      ]
    },
    {
      "credentials": {
        "httpHeaderAuth": {
          "id": "hr-platform",
          "name": "HR Platform API"
        }
      }
    },
    {
      "parameters": {
        "conditions": {
          "options": {
            "caseSensitive": true
          },
          "combinator": "and",
          "conditions": [
            {
              "leftValue": "={{ $json.summary.tasks }}",
              "rightValue": 0,
              "operator": {
                "type": "number",
                "operation": "gt"
              }
            }
          ]
        }
      }
    }
  ]
}
```

```

    }
  }
]
}
},
"id": "kasa-girişi-bekleniyor-mu",
"name": "Kasa girişi bekleniyor mu",
"type": "n8n-nodes-base.if",
"typeVersion": 2,
"position": [
  440,
  300
]
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ $env.HR_TICKETING_URL }}/tickets",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ JSON.stringify({tenant: $json.spec.tenant, title: '0. gün erişim paketi', queue: 'onboarding',
acceptance_criteria: 'Tüm erişimler kasaya yazıldı ve kanarya testi yeşil'}) }}",
    "authentication": "genericCredentialType",
    "genericAuthType": "httpHeaderAuth"
  },
  "id": "müşteri-erişim-formu",
  "name": "Müşteri erişim formu",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 1,
  "position": [
    660,
    200
  ],
  "credentials": {
    "httpHeaderAuth": {
      "id": "hr-platform",
      "name": "HR Platform API"
    }
  }
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ $env.HR_PLATFORM_URL }}/specs/apply",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ JSON.stringify({tenant: $json.tenant, entitlements: $json.entitlements, dry_run: false}) }}",
    "authentication": "genericCredentialType",
    "genericAuthType": "httpHeaderAuth"
  },
  "id": "kanarya-testi",
  "name": "Kanarya testi",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 1,
  "position": [
    880,
    200
  ],
  "credentials": {
    "httpHeaderAuth": {
      "id": "hr-platform",
      "name": "HR Platform API"
    }
  }
},
{
  "parameters": {
    "method": "POST",
    "url": "={{ $env.HR_TICKETING_URL }}/tickets",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ JSON.stringify({tenant: $json.tenant, title: 'Devreye alma durdu: kasa eksik', queue:
'onboarding', priority: 'P2'}) }}",
    "authentication": "genericCredentialType",
    "genericAuthType": "httpHeaderAuth"
  },

```

```
{
  "id": "başarısızsa-destek-kaydı",
  "name": "Başarısızsa destek kaydı",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 1,
  "position": [
    660,
    420
  ],
  "credentials": {
    "httpHeaderAuth": {
      "id": "hr-platform",
      "name": "HR Platform API"
    }
  }
},
"connections": {
  "Sözleşme imzalandı": {
    "main": [
      [
        {
          "node": "Erişim listesi üret",
          "type": "main",
          "index": 0
        }
      ]
    ],
    "Erişim listesi üret": {
      "main": [
        [
          {
            "node": "Kasa girişi bekleniyor mu",
            "type": "main",
            "index": 0
          }
        ]
      ],
      "Kasa girişi bekleniyor mu": {
        "main": [
          [
            {
              "node": "Müşteri erişim formu",
              "type": "main",
              "index": 0
            }
          ],
          [
            {
              "node": "Başarısızsa destek kaydı",
              "type": "main",
              "index": 0
            }
          ]
        ]
      ],
      "Müşteri erişim formu": {
        "main": [
          [
            {
              "node": "Kanarya testi",
              "type": "main",
              "index": 0
            }
          ]
        ]
      }
    }
  }
}
```

orchestrator/workflows/drift_reconcile.json

15 dakikada bir sapmaları uzlaştırır; etki sınırı aşılrısa geri alır

şerit: E – Takvimli iş · 205 satır · json

```
{
  "name": "drift_reconcile",
  "active": false,
  "settings": {
    "executionOrder": "v1"
  },
  "tags": [
    {
      "name": "drift"
    },
    {
      "name": "lane:E"
    }
  ],
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [
            {
              "field": "minutes",
              "minutesInterval": 15
            }
          ]
        }
      },
      "id": "her-15-dakikada",
      "name": "Her 15 dakikada",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [
        0,
        300
      ]
    },
    {
      "parameters": {
        "method": "GET",
        "url": "={{$env.HR_PLATFORM_URL}}/tenants",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth"
      },
      "id": "kiracıları-getir",
      "name": "Kiracıları getir",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 1,
      "position": [
        220,
        300
      ]
    },
    {
      "credentials": {
        "httpHeaderAuth": {
          "id": "hr-platform",
          "name": "HR Platform API"
        }
      }
    },
    {
      "parameters": {
        "batchSize": 1
      },
      "id": "her-kiracı-için",
      "name": "Her kiracı için",
      "type": "n8n-nodes-base.splitInBatches",
      "typeVersion": 3,
      "position": [
        440,
        300
      ]
    }
  ],
  {}
}
```

```

    "parameters": {
      "method": "POST",
      "url": "={{ $env.HR_PLATFORM_URL }}/specs/apply",
      "sendBody": true,
      "specifyBody": "json",
      "jsonBody": "={{ JSON.stringify({tenant: $json.slug, entitlements: $json.entitlements, dry_run: false}) }}",
      "authentication": "genericCredentialType",
      "genericAuthType": "httpHeaderAuth"
    },
    "id": "uzlaştır",
    "name": "Uzlaştır",
    "type": "n8n-nodes-base.httpRequest",
    "typeVersion": 1,
    "position": [
      660,
      300
    ],
    "credentials": {
      "httpHeaderAuth": {
        "id": "hr-platform",
        "name": "HR Platform API"
      }
    }
  },
  {
    "parameters": {
      "conditions": {
        "options": {
          "caseSensitive": true
        },
        "combinator": "and",
        "conditions": [
          {
            "leftValue": "={{ $json.error || '' }}",
            "rightValue": "BlastRadius",
            "operator": {
              "type": "string",
              "operation": "contains"
            }
          }
        ]
      }
    },
    "id": "etki-sınırı-aşıldıysa-dur",
    "name": "Etki sınırı aşıldıysa dur",
    "type": "n8n-nodes-base.if",
    "typeVersion": 2,
    "position": [
      880,
      300
    ]
  },
  {
    "parameters": {
      "method": "POST",
      "url": "={{ $env.HR_PLATFORM_URL }}/specs/rollback",
      "authentication": "genericCredentialType",
      "genericAuthType": "httpHeaderAuth"
    },
    "id": "son-sağlam-sürüme-dön",
    "name": "Son sağlam sürüme dön",
    "type": "n8n-nodes-base.httpRequest",
    "typeVersion": 1,
    "position": [
      1100,
      220
    ]
  },
  {
    "credentials": {
      "httpHeaderAuth": {
        "id": "hr-platform",
        "name": "HR Platform API"
      }
    }
  }
],
"connections": {

```

```
"Her 15 dakikada": {
  "main": [
    [
      {
        "node": "Kiracıları getir",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Kiracıları getir": {
  "main": [
    [
      {
        "node": "Her kiracı için",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Her kiracı için": {
  "main": [
    [
      {
        "node": "Uzlaştır",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Uzlaştır": {
  "main": [
    [
      {
        "node": "Etki sınırı aşıldıysa dur",
        "type": "main",
        "index": 0
      }
    ]
  ]
},
"Etki sınırı aşıldıysa dur": {
  "main": [
    [
      {
        "node": "Son sağlam sürüme dön",
        "type": "main",
        "index": 0
      }
    ],
    []
  ]
}
}
```

orchestrator/workflows/scheduled_monthly_report.json

Ayın 1'i 06:00'da aylık raporu üretir

şerit: E – Takvimli iş · 160 satır · json

```
{
  "name": "scheduled_monthly_report",
  "active": false,
  "settings": {
    "executionOrder": "v1"
  },
  "tags": [
    {
      "name": "scheduled"
    },
    {
      "name": "lane:E"
    }
  ],
  "nodes": [
    {
      "parameters": {
        "rule": {
          "interval": [
            {
              "field": "cronExpression",
              "expression": "0 6 1 * *"
            }
          ]
        },
        "timezone": "Europe/Istanbul"
      },
      "id": "ayın-1'i-06:00",
      "name": "Ayın 1'i 06:00",
      "type": "n8n-nodes-base.scheduleTrigger",
      "typeVersion": 1.2,
      "position": [
        0,
        300
      ]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "={{$env.HR_ZABBIX_URL}}",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth"
      },
      "id": "zabbix-erişilebilirlik",
      "name": "Zabbix erişilebilirlik",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 1,
      "position": [
        220,
        200
      ]
    },
    {
      "credentials": {
        "httpHeaderAuth": {
          "id": "zabbix",
          "name": "Zabbix API"
        }
      }
    },
    {
      "parameters": {
        "method": "GET",
        "url": "={{$env.HR_NETBOX_URL}}/api/dcim/devices/",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth"
      },
      "id": "netbox-envanter",
      "name": "NetBox envanter",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 1,
      "position": [
        220,
```

```

400
],
"credentials": {
  "httpHeaderAuth": {
    "id": "netbox",
    "name": "NetBox API"
  }
}
},
{
  "parameters": {
    "jsCode": "// Saat dilimi sorgularda sabitlenir: Europe/Istanbul ay sınırı ile\n// merkezin evrensel saat geçmiş i
ilk günü düşürebilir veya iki kez sayabilir.\nconst tz = 'Europe/Istanbul';\nreturn [{ json: { tz, availability: $('Zabbix
erişilebilirlik').all(), inventory: $('NetBox envanter').all() } }];"
  },
  "id": "raporu-derle",
  "name": "Raporu derle",
  "type": "n8n-nodes-base.code",
  "typeVersion": 2,
  "position": [
    440,
    300
  ]
},
{
  "parameters": {
    "method": "POST",
    "url": "={{$env.HR_REPORT_RENDER_URL}}/render"
  },
  "id": "pdf-üret-ve-gönder",
  "name": "PDF üret ve gönder",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 1,
  "position": [
    660,
    300
  ]
}
],
"connections": {
  "Ayın 1'i 06:00": {
    "main": [
      [
        {
          "node": "Zabbix erişilebilirlik",
          "type": "main",
          "index": 0
        },
        {
          "node": "NetBox envanter",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "Zabbix erişilebilirlik": {
    "main": [
      [
        {
          "node": "Raporu derle",
          "type": "main",
          "index": 0
        }
      ]
    ]
  },
  "NetBox envanter": {
    "main": [
      [
        {
          "node": "Raporu derle",
          "type": "main",
          "index": 0
        }
      ]
    ]
  }
}

```

```
]
},
"Raporu derle": {
  "main": [
    [
      {
        "node": "PDF üret ve gönder",
        "type": "main",
        "index": 0
      }
    ]
  ]
}
}
```

orchestrator/README.md

Akışların nasıl sürümlendiği ve sırların neden akışta durmadığı

şerit: E – Takvimli iş · 21 satır · markdown

```
# Orkestratör – n8n
```

İş akışları burada sürümlenir; n8n arayüzünden değil depodan yüklenir.

```
| Akış | Şerit | Ne yapar |
|-----|-----|-----|
| `onboarding.json` | D | Sözleşme imzalanınca erişim listesini üretir, formu açar, kanarya testini koşar |
| `drift_reconcile.json` | E | 15 dakikada bir sapmaları uzlaştırır; etki sınırı aşılsa geri alır |
| `scheduled_monthly_report.json` | E | Ayın 1'i 06:00 (Europe/Istanbul) aylık raporu üretir |
```

```
### Sırlar
```

Hiçbir akışta açık metin parola bulunmaz. Her düğüm **n8n credentials** kaydına referans verir (`credentials` alanı). Dışa aktarırken bu alan yalnız kimlik ve ad taşır; değer in kendisi n8n'in şifreli deposunda kalır.

```
```bash
İçerik aktarma
n8n import:workflow --separate --input=workflows/
```
```

2 · HR Platformu

Beyin: katalog, tanım derleyici, şerit dağıtıcısı, entegrasyonlar

platform/app/main.py

Giriş noktası; katalog tutarsızsa açılışta patlar

şerit: D – Platform kurulumu · 29 satır · python

```
"""HR Platformu – giriş noktası."""
from __future__ import annotations

import structlog
from fastapi import FastAPI

from app.api.routers import agents, catalog, health, specs
from app.catalog import load_catalog
from app.config import settings

log = structlog.get_logger(__name__)

app = FastAPI(
    title="Hisar Delivery Platform",
    version="0.1.0",
    description="Katalog, hak seti, tanım derleyici ve şerit dağıtıcısı",
)

app.include_router(health.router)
app.include_router(catalog.router)
app.include_router(specs.router)
app.include_router(agents.router)

@app.on_event("startup")
async def startup() -> None:
    cat = load_catalog(settings().catalog_dir) # tutarsızlık varsa açılışta patlar
    log.info("katalog_yuklendi", hizmet=len(cat))
```

platform/app/config.py

Tüm sırlar ortam değişkeninden; kodda sabit değer yok

şerit: D – Platform kurulumu · 35 satır · python

```
"""Yapılandırma. Tüm sırlar ortam değişkeninden gelir; kodda sabit değer bulunmaz."""
from functools import lru_cache
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="HR_", env_file=".env", extra="ignore")

    database_url: str = "postgresql+asyncpg://hisar:hisar@localhost:5432/hisar"

    netbox_url: str = ""
    netbox_token: str = Field(default="", repr=False)
    zabbix_url: str = ""
    zabbix_token: str = Field(default="", repr=False)
    awx_url: str = ""
    awx_token: str = Field(default="", repr=False)
    siem_url: str = ""
    siem_token: str = Field(default="", repr=False)
    ticketing_url: str = ""
    ticketing_token: str = Field(default="", repr=False)
    vault_addr: str = ""
    vault_token: str = Field(default="", repr=False)

    agent_enroll_secret: str = Field(default="", repr=False)
    agent_checkin_seconds: int = 45
    drift_interval_seconds: int = 900

    catalog_dir: str = "app/catalog/services"

@lru_cache
def settings() -> Settings:
    return Settings()
```

platform/app/models.py

Kiracı, hak seti, cihaz, sürümlü tanım, görev koşusu, denetim kaydı

şerit: D – Platform kurulumu · 107 satır · python

```
"""Kalıcı veri modeli."""
from __future__ import annotations

import enum
from datetime import datetime
from typing import List, Optional

from sqlalchemy import JSON, DateTime, Enum, ForeignKey, Integer, String, UniqueConstraint, func
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship


class Base(DeclarativeBase):
    pass


class Lane(str, enum.Enum):
    A = "A" # uç cihaz ajanı
    B = "B" # Ansible ile uzaktan ayar
    C = "C" # sağlayıcı arayüzü
    D = "D" # HR platform kurulumu
    E = "E" # takvimli iş
    F = "F" # insan iş emri


class TaskState(str, enum.Enum):
    PENDING = "pending"
    BLOCKED = "blocked" # ön koşulu tamamlanmadı
    RUNNING = "running"
    DONE = "done"
    FAILED = "failed"
    SKIPPED = "skipped"


class Tenant(Base):
    __tablename__ = "tenants"
    id: Mapped[int] = mapped_column(primary_key=True)
    slug: Mapped[str] = mapped_column(String(64), unique=True)
    name: Mapped[str] = mapped_column(String(200))
    package: Mapped[str] = mapped_column(String(32)) # Essential | Professional | Enterprise
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

    entitlements: Mapped[List["Entitlement"]] = relationship(back_populates="tenant")
    devices: Mapped[List["Device"]] = relationship(back_populates="tenant")


class Entitlement(Base):
    """Müşterinin satın aldığı hizmetlerin listesi. Sitedeki hizmet kimliğiyle aynıdır."""
    __tablename__ = "entitlements"
    __table_args__ = (UniqueConstraint("tenant_id", "service_id"),)

    id: Mapped[int] = mapped_column(primary_key=True)
    tenant_id: Mapped[int] = mapped_column(ForeignKey("tenants.id"))
    service_id: Mapped[str] = mapped_column(String(80))
    active: Mapped[bool] = mapped_column(default=True)

    tenant: Mapped[Tenant] = relationship(back_populates="entitlements")


class Device(Base):
    __tablename__ = "devices"
    id: Mapped[int] = mapped_column(primary_key=True)
    tenant_id: Mapped[int] = mapped_column(ForeignKey("tenants.id"))
    uuid: Mapped[str] = mapped_column(String(64), unique=True) # agent kimliği
    serial: Mapped[Optional[str]] = mapped_column(String(120), default=None)
    hostname: Mapped[str] = mapped_column(String(200))
    role: Mapped[str] = mapped_column(String(64)) # windows-server, switch, ...
    os_family: Mapped[Optional[str]] = mapped_column(String(32), default=None)
    agent_capable: Mapped[bool] = mapped_column(default=True)
    last_seen: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), default=None)

    tenant: Mapped[Tenant] = relationship(back_populates="devices")
```

```

class SpecVersion(Base):
    """Derlenmiş arzu edilen durum. Sürümlüdür; geri alma buna dayanır."""
    __tablename__ = "spec_versions"
    id: Mapped[int] = mapped_column(primary_key=True)
    tenant_id: Mapped[int] = mapped_column(ForeignKey("tenants.id"))
    version: Mapped[int] = mapped_column(Integer)
    payload: Mapped[dict] = mapped_column(JSON)
    known_good: Mapped[bool] = mapped_column(default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

class TaskRun(Base):
    __tablename__ = "task_runs"
    id: Mapped[int] = mapped_column(primary_key=True)
    tenant_id: Mapped[int] = mapped_column(ForeignKey("tenants.id"))
    spec_version: Mapped[int] = mapped_column(Integer)
    service_id: Mapped[str] = mapped_column(String(80))
    task_id: Mapped[str] = mapped_column(String(120))
    lane: Mapped[Lane] = mapped_column(Enum(Lane))
    state: Mapped[TaskState] = mapped_column(Enum(TaskState), default=TaskState.PENDING)
    detail: Mapped[Optional[dict]] = mapped_column(JSON, default=None)
    started_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), default=None)
    finished_at: Mapped[Optional[datetime]] = mapped_column(DateTime(timezone=True), default=None)

class AuditEvent(Base):
    __tablename__ = "audit_events"
    id: Mapped[int] = mapped_column(primary_key=True)
    tenant_id: Mapped[Optional[int]] = mapped_column(ForeignKey("tenants.id"), default=None)
    actor: Mapped[str] = mapped_column(String(120))
    action: Mapped[str] = mapped_column(String(120))
    payload: Mapped[Optional[dict]] = mapped_column(JSON, default=None)
    at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())

```

platform/app/catalog/loader.py

Katalog yükleyici; 'ilk görev kasa olmalı' kuralını zorunlu kılar

şerit: D – Platform kurulumu · 143 satır · python

```
"""Katalog yükleyici.

Tek katalog, iki yüz: pre-sales.hisarresearch.com'daki hizmet kimliği ile buradaki
'id' aynı olmak zorundadır. Farklılaşırsa satılan ile teslim edilen ayrışır.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path

import yaml

from app.models import Lane

VALID_GRADES = {"FULL", "ASSISTED", "HUMAN"}
VALID_STORES = {".env", "ansible-vault", "n8n-credentials"}


class CatalogError(Exception):
    """Katalog tutarsızlığı – açılışta patlar, çalışma anında değil."""


@dataclass(frozen=True)
class Task:
    id: str
    lane: Lane
    action: str
    description: str
    params: dict = field(default_factory=dict)
    requires_secret: list[str] = field(default_factory=list)


@dataclass(frozen=True)
class CatalogService:
    id: str
    name: str
    area: str
    tier: str
    lane: Lane
    lanes: tuple[Lane, ...]
    plain: str
    executor: str
    automation_grade: str
    secret_stores: tuple[str, ...]
    requires: tuple[str, ...]
    tasks: tuple[Task, ...]
    health_check: str
    gotcha: str
    teardown: tuple[Task, ...]

    @property
    def first_task_is_vault(self) -> bool:
        """Her hizmetin ilk adımı erişim bilgilerinin kasaya yazılmasıdır."""
        return bool(self.tasks) and self.tasks[0].action == "vault.store_access"


@dataclass
class Catalog:
    services: dict[str, CatalogService]

    def __iter__(self):
        return iter(self.services.values())

    def __len__(self) -> int:
        return len(self.services)

    def get(self, service_id: str) -> CatalogService:
        try:
            return self.services[service_id]
        except KeyError as exc:
            raise CatalogError(f"katalogda yok: {service_id}") from exc
```

```

def by_lane(self, lane: Lane) -> list[CatalogService]:
    return [s for s in self.services.values() if s.lane is lane]

def _task(raw: dict, where: str) -> Task:
    for key in ("id", "lane", "action"):
        if key not in raw:
            raise CatalogError(f"{where}: görevde '{key}' eksik")
    return Task(
        id=raw["id"],
        lane=Lane(raw["lane"]),
        action=raw["action"],
        description=raw.get("description", ""),
        params=raw.get("params", {}) or {},
        requires_secret=list(raw.get("requires_secret", []) or []),
    )

def load_catalog(directory: str | Path) -> Catalog:
    path = Path(directory)
    if not path.is_dir():
        raise CatalogError(f"katalog klasörü yok: {path}")

    services: dict[str, CatalogService] = {}
    for file in sorted(path.glob("*.yaml")):
        raw = yaml.safe_load(file.read_text(encoding="utf-8"))
        sid = raw.get("id")
        if not sid:
            raise CatalogError(f"{file.name}: 'id' eksik")
        if sid in services:
            raise CatalogError(f"{file.name}: yinelenen kimlik {sid}")
        if raw.get("automation_grade") not in VALID_GRADES:
            raise CatalogError(f"{sid}: geçersiz automation_grade")

        stores = tuple(raw.get("secret_stores", []) or [])
        unknown = set(stores) - VALID_STORES
        if unknown:
            raise CatalogError(f"{sid}: bilinmeyen kasa {unknown}")

        tasks = tuple(_task(t, sid) for t in raw.get("tasks", []))
        service = CatalogService(
            id=sid,
            name=raw["name"],
            area=raw.get("area", ""),
            tier=raw.get("tier", "Proje"),
            lane=Lane(raw["lane"]),
            lanes=tuple(Lane(x) for x in raw.get("lanes", [raw["lane"]])),
            plain=raw.get("plain", ""),
            executor=raw.get("executor", ""),
            automation_grade=raw["automation_grade"],
            secret_stores=stores,
            requires=tuple(raw.get("requires", []) or []),
            tasks=tasks,
            health_check=raw.get("health_check", ""),
            gotcha=raw.get("gotcha", ""),
            teardown=tuple(_task(t, sid) for t in raw.get("teardown", []) or []),
        )
        if not service.first_task_is_vault:
            raise CatalogError(
                f"{sid}: ilk görev 'vault.store_access' olmalı - "
                "kasaya yazılmamış yetkiyle hizmet başlatılmaz"
            )
        services[sid] = service

    # ön koşullar katalogda var mı
    for svc in services.values():
        for dep in svc.requires:
            if dep not in services:
                raise CatalogError(f"{svc.id}: tanımsız ön koşul {dep}")
    return Catalog(services=services)

```

platform/app/services/task_graph.py

Ön koşullara göre sıralama; envanter her şeyden önce gelir

şerit: D – Platform kurulumu · 52 satır · python

```
"""Görev grafiği: hizmetleri ön koşullarına göre sıralar.

Bir hizmetin görevleri, bağlı olduğu hizmetlerin görevleri bittikten sonra sıraya girer.
Böylece Zabbix host'u, türetildiği NetBox kaydından önce oluşturulamaz.
"""

from __future__ import annotations

from collections import deque

from app.catalog import Catalog, CatalogService

class DependencyCycle(Exception):
    pass

def topological_order(catalog: Catalog, service_ids: list[str]) -> list[CatalogService]:
    """Verilen hizmetleri bağımlılık sırasına dizer (kararlı: alfabetik ikincil sıra)."""
    wanted = set(service_ids)

    # kapanış: ön koşullar hak setinde olmasa bile kurulmalıdır
    queue = deque(wanted)
    while queue:
        sid = queue.popleft()
        for dep in catalog.get(sid).requires:
            if dep not in wanted:
                wanted.add(dep)
                queue.append(dep)

    indegree = {sid: 0 for sid in wanted}
    dependents: dict[str, list[str]] = {sid: [] for sid in wanted}
    for sid in wanted:
        for dep in catalog.get(sid).requires:
            indegree[sid] += 1
            dependents[dep].append(sid)

    ready = sorted([sid for sid, deg in indegree.items() if deg == 0])
    order: list[str] = []
    while ready:
        sid = ready.pop(0)
        order.append(sid)
        for nxt in sorted(dependents[sid]):
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                ready.append(nxt)
                ready.sort()

    if len(order) != len(wanted):
        stuck = sorted(set(wanted) - set(order))
        raise DependencyCycle(f"ön koşul döngüsü: {stuck}")

    return [catalog.get(sid) for sid in order]
```

platform/app/services/spec_compiler.py

Hak seti + cihaz rolü → arzu edilen durum; çıkarmalı çalışır

şerit: D – Platform kurulumu · 120 satır · python

```
"""Tanım derleyici: hak seti + cihaz rolleri → arzu edilen durum.

Çıktı sürümlüdür ve **çıkarmalıdır**: paket düşünce kaldırılması gereken hizmetler
söküm görevleriyle sıraya girer. Böylece paket düşürme ayrı bir proje olmaktan çıkar.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass, field

from app.catalog import Catalog, CatalogService, Task
from app.models import Lane
from app.services.task_graph import topological_order

class MissingSecret(Exception):
    """Kasaya yazılmamış bir yetkiyle hizmet başlatılamaz."""

@dataclass
class CompiledTask:
    service_id: str
    task_id: str
    lane: str
    action: str
    description: str
    params: dict = field(default_factory=dict)
    teardown: bool = False

@dataclass
class CompiledSpec:
    tenant: str
    version: int
    tasks: list[CompiledTask]
    services: list[str]
    removed: list[str]

    def to_json(self) -> dict:
        return {
            "tenant": self.tenant,
            "version": self.version,
            "services": self.services,
            "removed": self.removed,
            "tasks": [asdict(t) for t in self.tasks],
        }

    def by_lane(self, lane: Lane) -> list[CompiledTask]:
        return [t for t in self.tasks if t.lane == lane.value]

def _expand(svc: CatalogService, device_roles: set[str]) -> list[Task]:
    """Cihaz rolüne göre geçerli olmayan görevleri eler."""
    out = []
    for task in svc.tasks:
        only = task.params.get("only_roles")
        if only and not (set(only) & device_roles):
            continue
        out.append(task)
    return out

def compile_spec(
    *,
    catalog: Catalog,
    tenant: str,
    entitlements: list[str],
    previous_services: list[str] | None = None,
    device_roles: set[str] | None = None,
    version: int = 1,
    vault_has: set[str] | None = None,
) -> CompiledSpec:
```

```

device_roles = device_roles or set()
previous = set(previous_services or [])
vault_has = vault_has if vault_has is not None else None

ordered = topological_order(catalog, entitlements)
tasks: list[CompiledTask] = []

for svc in ordered:
    for task in _expand(svc, device_roles):
        if vault_has is not None:
            missing = [s for s in task.requires_secret if s not in vault_has]
            if missing:
                raise MissingSecret(f"{svc.id}/{task.id}: kasada yok → {missing}")
        tasks.append(
            CompiledTask(
                service_id=svc.id,
                task_id=task.id,
                lane=task.lane.value,
                action=task.action,
                description=task.description,
                params=task.params,
            )
        )

# çıkarmalı taraf: artık hak edilmeyen hizmetlerin sökümü
current = {s.id for s in ordered}
removed = sorted(previous - current)
for sid in removed:
    for task in catalog.get(sid).teardown:
        tasks.append(
            CompiledTask(
                service_id=sid,
                task_id=task.id,
                lane=task.lane.value,
                action=task.action,
                description=task.description,
                params=task.params,
                teardown=True,
            )
        )

return CompiledSpec(
    tenant=tenant,
    version=version,
    tasks=tasks,
    services=[s.id for s in ordered],
    removed=removed,
)

```

platform/app/services/dispatcher.py

Her görevi kendi şeridinin yürütücüsüne yollar — altı şeridin bulunduğu yer

şerit: A – Aracı yazılım B – Uzaktan ayar C – Sağlayıcı arayüzü D – Platform kurulumu E – Takvimli iş F – İnsan işi · 119
satır · python

```
"""Dağıtıcı: her görevi kendi şeridinin yürütücüsüne yollar.

Şerit → yürütücü eşlemesi tek yerde durur. Yeni bir şerit eklenirse burası genişler,
katalog dosyalarının hiçbirisi değişmez.
"""
from __future__ import annotations

import structlog

from app.integrations import awx, netbox, siem, ticketing, vault, zabbix
from app.models import Lane
from app.services.spec_compiler import CompiledSpec, CompiledTask

log = structlog.get_logger(__name__)

class LaneNotImplemented(Exception):
    pass

async def _lane_a(task: CompiledTask, tenant: str) -> dict:
    """Uç cihaz ajanı: görev kuyruğa yazılır, ajan bir sonraki yoklamasında çeker."""
    from app.services import agent_queue

    return await agent_queue.enqueue(tenant=tenant, task=task)

async def _lane_b(task: CompiledTask, tenant: str) -> dict:
    """Ansible: AWX iş şablonu, müşterinin yürütme düğümünde koşar."""
    return await awx.launch_job(
        job_template=task.params.get("job_template", task.action.split(".")[1]),
        instance_group=f"relay-{tenant}",
        extra_vars={"tenant": tenant, **task.params.get("extra_vars", {})},
    )

async def _lane_c(task: CompiledTask, tenant: str) -> dict:
    """Sağlayıcı arayüzü: ayar müşteride değil, sağlayıcının sisteminde."""
    provider, _, op = task.action.partition(".")
    handler = {
        "netbox": netbox.call,
        "zabbix": zabbix.call,
        "veeam": awx.call_via_relay, # Veeam API'si müşteri ağında → relay üzerinden
        "cloud": awx.call_via_relay,
    }.get(provider)
    if handler is None:
        raise LaneNotImplemented(f"C şeridinde bilinmeyen sağlayıcı: {provider}")
    return await handler(op, tenant=tenant, **task.params)

async def _lane_d(task: CompiledTask, tenant: str) -> dict:
    """HR platform kurulumu: iş bizim tarafımızda."""
    target, _, op = task.action.partition(".")
    handler = {
        "netbox": netbox.call,
        "zabbix": zabbix.call,
        "siem": siem.call,
        "ticketing": ticketing.call,
        "vault": vault.call,
    }.get(target)
    if handler is None:
        raise LaneNotImplemented(f"D şeridinde bilinmeyen hedef: {target}")
    return await handler(op, tenant=tenant, **task.params)

async def _lane_e(task: CompiledTask, tenant: str) -> dict:
    """Takvimli iş: n8n'de zamanlama kurulur, sonra kendi takviminde koşar."""
    from app.integrations import n8n

    return await n8n.ensure_schedule(
```

```

        workflow=task.params["workflow"], tenant=tenant, cron=task.params.get("cron", "0 6 1 * *")
    )

async def _lane_f(task: CompiledTask, tenant: str) -> dict:
    """İnsan iş emri: otomasyon hakkı kurar, teslimi insan yapar."""
    return await ticketing.create_work_order(
        tenant=tenant,
        title=task.description or task.task_id,
        queue=task.params.get("queue", "saha"),
        sla=task.params.get("sla"),
        acceptance=task.params.get("acceptance"),
    )

LANES = {
    Lane.A: _lane_a,
    Lane.B: _lane_b,
    Lane.C: _lane_c,
    Lane.D: _lane_d,
    Lane.E: _lane_e,
    Lane.F: _lane_f,
}

async def dispatch(spec: CompiledSpec, *, dry_run: bool = False) -> list[dict]:
    """Derlenmiş tanımı sırayla yürütür. Sıra zaten görev grafiğinden gelir."""
    results: list[dict] = []
    for task in spec.tasks:
        lane = Lane(task.lane)
        entry = {
            "service": task.service_id,
            "task": task.task_id,
            "lane": task.lane,
            "teardown": task.teardown,
        }
        if dry_run:
            results.append({**entry, "state": "dry-run"})
            continue
        try:
            entry["result"] = await LANES[lane](task, spec.tenant)
            entry["state"] = "done"
        except Exception as exc: # noqa: BLE001 – tek görev hatası tüm tanımı düşürmez
            log.error("gorev_hatasi", service=task.service_id, task=task.task_id, error=str(exc))
            entry["state"] = "failed"
            entry["error"] = str(exc)
        results.append(entry)
    return results

```

platform/app/services/agent_queue.py

A şeridi: görev relay kuyruğuna yazılır, ajan çeker

şerit: A – Aracı yazılım · 36 satır · python

```
"""A şeridi kuyruğu.

Platform ajanı hiçbir zaman aramaz. Görev, müşterinin site relay'indeki kuyruğa yazılır;
ajan bir sonraki dışa doğru yoklamasında onu çeker. Yön her zaman içeriden dışarıdır.
"""

from __future__ import annotations

import httpx
import structlog

from app.config import settings
from app.services.spec_compiler import CompiledTask

log = structlog.get_logger(__name__)

async def enqueue(*, tenant: str, task: CompiledTask, device_uuid: str | None = None) -> dict:
    """Relay'in iş kuyruğuna görev bırakır."""
    payload = {
        "service_id": task.service_id,
        "task_id": task.task_id,
        "action": task.action,
        "params": task.params,
        "device_uuid": device_uuid,
        "teardown": task.teardown,
    }
    url = f"{settings().awx_url}/relay/{tenant}/queue" # AWX mesh üzerinden relay'e
    async with httpx.AsyncClient(timeout=20) as client:
        resp = await client.post(url, json=payload, headers=_auth())
        resp.raise_for_status()
        return resp.json()

def _auth() -> dict[str, str]:
    return {"Authorization": f"Bearer {settings().awx_token}"}
```

platform/app/services/drift.py

Sapma uzlaştırma ve etki sınırı koruması

şerit: A – Aracı yazılım B – Uzaktan ayar C – Sağlayıcı arayüzü D – Platform kurulumu · 64 satır · python

```
"""Sapma uzlaştırması.

Devreye alma ile günlük işletme aynı döngüdür: ilk gün ne koşuyorsa 400. gün de o koşar.
Fark yalnızca şudur – ilk gün her şey eksiktir, sonra yalnız sapanlar uygulanır.
"""

from __future__ import annotations

import structlog

from app.services.dispatcher import dispatch
from app.services.spec_compiler import CompiledSpec, CompiledTask

log = structlog.get_logger(__name__)

# hatalı bir tanımın tek seferde dokunabileceği en fazla cihaz oranı
BLAST_RADIUS = 0.25

class BlastRadiusExceeded(Exception):
    """Bir uygulama tüm sahayı değil, en fazla belirlenen dilimi etkileyebilir."""

def diff(spec: CompiledSpec, observed: dict[str, str]) -> list[CompiledTask]:
    """Arzu edilen durum ile bildirilen durumu karşılaştırır, sapan görevleri döndürür."""
    out = []
    for task in spec.tasks:
        key = f"{task.service_id}/{task.task_id}"
        if observed.get(key) != "ok":
            out.append(task)
    return out

async def reconcile(
    spec: CompiledSpec,
    observed: dict[str, str],
    *,
    device_count: int = 1,
    dry_run: bool = False,
) -> dict:
    drifted = diff(spec, observed)
    if not drifted:
        return {"drifted": 0, "applied": [], "state": "in-sync"}

    if device_count and len(drifted) / max(device_count, 1) > BLAST_RADIUS and not dry_run:
        raise BlastRadiusExceeded(
            f"{len(drifted)} görev {device_count} cihazın %{BLAST_RADIUS:.0%} sınırını aşıyor"
        )

    subset = CompiledSpec(
        tenant=spec.tenant,
        version=spec.version,
        tasks=drifted,
        services=spec.services,
        removed=spec.removed,
    )

    applied = await dispatch(subset, dry_run=dry_run)
    failed = [a for a in applied if a.get("state") == "failed"]
    return {
        "drifted": len(drifted),
        "applied": applied,
        "state": "repaired" if not failed else "partial",
        "rollback_hint": "son sağlam sürüme dön" if failed else None,
    }
```

platform/app/security.py

Ajan kimliği ve kiracı sınırı

şerit: A – Aracı yazılım · 59 satır · python

```
"""Ajan kimliği ve kiracı sınırı.

Kasa, her müşterinin sırlarını tutar. Acme'nin ajanı, Contoso'nun tanımını isteyememelidir;
bu yüzden her istek cihaz kimliğine bağlı bir belirteçle imzalanır ve kiracı sunucuda çözülür.
"""

from __future__ import annotations

import hashlib
import hmac
import secrets
from datetime import datetime, timedelta, timezone

from fastapi import Header, HTTPException

from app.config import settings

TOKEN_TTL = timedelta(hours=12)

SEP = "|" # ISO zaman damgası iki nokta içerdiği için ayraç olarak ':' kullanılamaz


def issue_agent_token(*, tenant: str, device_uuid: str) -> str:
    """Kayıt sırasında cihaza özel belirteç üretir."""
    if SEP in tenant or SEP in device_uuid:
        raise ValueError(f"kiracı veya cihaz kimliğinde ayraç karakteri: {SEP}")
    issued = datetime.now(timezone.utc).isoformat()
    body = SEP.join((tenant, device_uuid, issued))
    mac = hmac.new(settings().agent_enroll_secret.encode(), body.encode(), hashlib.sha256)
    return SEP.join((body, mac.hexdigest()))


def verify_agent_token(token: str) -> tuple[str, str]:
    """(tenant, device_uuid) döndürür; geçersizse 401."""
    try:
        tenant, device_uuid, issued, mac = token.split(SEP)
    except ValueError:
        raise HTTPException(401, "bozuk belirteç") from None

    body = SEP.join((tenant, device_uuid, issued))
    expected = hmac.new(
        settings().agent_enroll_secret.encode(), body.encode(), hashlib.sha256
    ).hexdigest()
    if not hmac.compare_digest(mac, expected):
        raise HTTPException(401, "imza doğrulanamadı")
    if datetime.fromisoformat(issued) + TOKEN_TTL < datetime.now(timezone.utc):
        raise HTTPException(401, "belirteç süresi doldu")
    return tenant, device_uuid


async def agent_identity(authorization: str = Header(default="")) -> tuple[str, str]:
    if not authorization.startswith("Bearer "):
        raise HTTPException(401, "yetki başlığı yok")
    return verify_agent_token(authorization.removeprefix("Bearer ").strip())


def new_enrollment_code() -> str:
    return secrets.token_urlsafe(24)
```

platform/app/integrations/awx.py

B şeridinin yürütücüsü; relay'i dışa doğru eşleşen yürütme düğümü olarak kaydeder

şerit: B – Uzaktan ayar · 53 satır · python

```
"""AWX – B şeridinin yürütücüsü.

Relay, n8n worker'ı DEĞİLDİR. AWX automation mesh'te bir **yürütme düğümüdür**:
`listener_port: null` ve `peers_from_control_nodes: false` ile yapılandırılır, yani
bağlantıyı kendisi dışarı doğru kurar. Müşteri güvenlik duvarında gelen kural açılmaz.
Ayrıntı: docs/architecture.md
"""

from __future__ import annotations

from app.config import settings
from app.integrations._client import request

def _url(path: str) -> str:
    return f"{settings().awx_url.rstrip('/')}/api/v2/{path.lstrip('/')}"

async def launch_job(*, job_template: str, instance_group: str, extra_vars: dict) -> dict:
    """İşi müşterinin yürütme düğümünde koşturur."""
    return await request(
        "POST", _url(f"job_templates/{job_template}/launch/"),
        token=settings().awx_token,
        json={"extra_vars": extra_vars, "instance_group": instance_group},
    )

async def job_status(*, job_id: int) -> dict:
    return await request("GET", _url(f"jobs/{job_id}/"), token=settings().awx_token)

async def call_via_relay(op: str, *, tenant: str, **params) -> dict:
    """Müşteri ağındaki arayüze (Veeam, vCenter) relay üzerinden ulaşır."""
    return await launch_job(
        job_template="relay_api_call",
        instance_group=f"relay-{tenant}",
        extra_vars={"operation": op, "tenant": tenant, **params},
    )

async def ensure_execution_node(*, tenant: str, hostname: str) -> dict:
    """Yeni müşteri relay'ini mesh'e kaydeder – dışa doğru eşleşme ile."""
    return await request(
        "POST", _url("instances/"),
        token=settings().awx_token,
        json={
            "hostname": hostname,
            "node_type": "execution",
            "listener_port": None, # gelen bağlantı dinlemez
            "peers_from_control_nodes": False, # bağlantıyı düğüm kurar
            "managed_by_policy": False,
        },
    )
```

platform/app/integrations/vault.py

Kasa; kiracı ayrışması yol düzeyinde zorunlu

şerit: D – Platform kurulumu · 58 satır · python

```
"""Kasa – her hizmetin ilk adımı buraya yazmaktır.

Kiracı ayrışması yol düzeyinde zorunludur: `tenants/<slug>/...`. Bir müşteri için
çalışan iş akışı başka müşterinin yolunu isterse kasa reddeder.
"""

from __future__ import annotations

from app.config import settings
from app.integrations._client import request

class CrossTenantAccess(Exception):
    """A kiracısı için çalışan iş, B kiracısının alanını isteyemez."""

def _path(tenant: str, key: str) -> str:
    if key.startswith("tenants/") and not key.startswith(f"tenants/{tenant}/"):
        raise CrossTenantAccess(f"{tenant} → {key}")
    return f"tenants/{tenant}/{key.lstrip('/')}"

async def call(op: str, *, tenant: str, **params) -> dict:
    return await {"store_access": store_access, "has": has, "revoke": revoke}[op](
        tenant=tenant, **params
    )

async def store_access(*, tenant: str, keys: list[str] | None = None, store: str = "", **_) -> dict:
    """Erişim bilgilerini ilgili kasaya yazar.

    `store` hangi kasanın kullanılacağını söyler: `.env`, `ansible-vault`
    veya `n8n-credentials`. Değer platforma hiç uğramaz; yalnız referansı tutulur.
    """
    return await request(
        "POST", f"{settings().vault_addr.rstrip('/')}/v1/secret/data/{_path(tenant, 'access')}",
        token=settings().vault_token, header="X-Vault-Token", prefix="",
        json={"data": {"store": store, "keys": keys or [], "status": "pending-intake"}},
    )

async def has(*, tenant: str, key: str, **_) -> bool:
    try:
        await request(
            "GET", f"{settings().vault_addr.rstrip('/')}/v1/secret/data/{_path(tenant, key)}",
            token=settings().vault_token, header="X-Vault-Token", prefix="",
        )
        return True
    except RuntimeError:
        return False

async def revoke(*, tenant: str, key: str = "", **_) -> dict:
    """Çıkış sürecinin parçası: anahtar iptal edilir ve değiştirilir."""
    return await request(
        "DELETE", f"{settings().vault_addr.rstrip('/')}/v1/secret/metadata/{_path(tenant, key)}",
        token=settings().vault_token, header="X-Vault-Token", prefix="",
    )
```

platform/app/integrations/netbox.py

Envanter; kimlik anahtarı ajan kimliği + seri no

şerit: C – Sağlayıcı arayüzü D – Platform kurulumu · 62 satır · python

```
"""NetBox – envanter ve doğruluk kaynağı.

Her şeyin sağlık kontrolü buna karşı çözümlenir: 'var olanı envanterle karşılaştır'.
Kimlik anahtarı hostname veya ağ kartı kimliği DEĞİL, ajan kimliği + seri numarasıdır –
klonlanmış sanal makineler ikisini de paylaşır.
"""

from __future__ import annotations

from app.config import settings
from app.integrations._client import request

def _url(path: str) -> str:
    return f"{settings().netbox_url.rstrip('/')}/api/{path.lstrip('/')}"

async def call(op: str, *, tenant: str, **params) -> dict:
    return await {
        "ensure_tenant": ensure_tenant,
        "ensure_device": ensure_device,
        "decommission": decommission,
        "list_devices": list_devices,
    }[op](tenant=tenant, **params)

async def ensure_tenant(*, tenant: str, **_) -> dict:
    return await request(
        "POST", _url("tenancy/tenants/"),
        token=settings().netbox_token, prefix="Token",
        json={"name": tenant, "slug": tenant},
    )

async def ensure_device(*, tenant: str, uuid: str, serial: str | None = None, **fields) -> dict:
    """Ajan kimliği birincil anahtardır; seri numarası ikincil doğrulamadır."""
    return await request(
        "POST", _url("dcim/devices/"),
        token=settings().netbox_token, prefix="Token",
        json={
            "tenant": {"slug": tenant},
            "custom_fields": {"agent_uuid": uuid},
            "serial": serial or "",
            **fields,
        },
    )

async def list_devices(*, tenant: str, **_) -> dict:
    return await request(
        "GET", _url(f"dcim/devices/?tenant={tenant}&limit=1000"),
        token=settings().netbox_token, prefix="Token",
    )

async def decommission(*, tenant: str, uuid: str, **_) -> dict:
    """Söküm: kayıt silinmez, durumu değişir – envanter geçmişini korunur."""
    return await request(
        "PATCH", _url(f"dcim/devices/?cf_agent_uuid={uuid}"),
        token=settings().netbox_token, prefix="Token",
        json={"status": "decommissioning", "tenant": {"slug": tenant}},
    )
```

platform/app/integrations/zabbix.py

İzleme; satılan şey host kaydı ve uyarı kuralıdır

şerit: D – Platform kurulumu · 55 satır · python

```
"""Zabbix – izleme. D şeridinin ana yükü buradadır.

Ajanın kurulması taşımadır; satılan şey host kaydı, şablon bağı ve uyarı kuralıdır.
A şeridi kusursuz çalışıp D şeridi hiç koşturmasa müşteri hiçbir şey satın almamış olur.
"""

from __future__ import annotations

from app.config import settings
from app.integrations._client import request

async def _rpc(method: str, params: dict) -> dict:
    return await request(
        "POST", settings().zabbix_url,
        json={"jsonrpc": "2.0", "method": method, "params": params, "id": 1},
        headers={"Authorization": f"Bearer {settings().zabbix_token}"},
    )

async def call(op: str, *, tenant: str, **params) -> dict:
    return await {
        "ensure_host_group": ensure_host_group,
        "ensure_host": ensure_host,
        "link_template": link_template,
        "ensure_action": ensure_action,
    }[op](tenant=tenant, **params)

async def ensure_host_group(*, tenant: str, **_) -> dict:
    return await _rpc("hostgroup.create", {"name": f"tenant/{tenant}"})

async def ensure_host(*, tenant: str, host: str, proxy: str | None = None, **_) -> dict:
    """Host her zaman müşterinin site relay proxy'sine bağlanır – doğrudan değil."""
    return await _rpc(
        "host.create",
        {
            "host": host,
            "groups": [{"name": f"tenant/{tenant}"}],
            "proxy_hostid": proxy or f"relay-{tenant}",
            "tags": [{"tag": "tenant", "value": tenant}],
        },
    )

async def link_template(*, tenant: str, host: str, template: str, **_) -> dict:
    return await _rpc("host.update", {"host": host, "templates": [{"name": template}]})

async def ensure_action(*, tenant: str, name: str, escalation: list | None = None, **_) -> dict:
    return await _rpc(
        "action.create",
        {"name": f"{tenant}/{name}", "operations": escalation or [], "status": 0},
    )
```

platform/app/integrations/ticketing.py

F şeridinin arayüzü; kabul kriteri olmadan iş emri kapanmaz

şerit: F – İnsan işi · 58 satır · python

```
"""Destek kaydı sistemi – F şeridinin arayüzü ve paketlerin şekillendiği yer."""
from __future__ import annotations

from app.config import settings
from app.integrations._client import request

async def call(op: str, *, tenant: str, **params) -> dict:
    return await {
        "ensure_tenant": ensure_tenant,
        "ensure_queue": ensure_queue,
        "ensure_sla": ensure_sla,
        "create_work_order": create_work_order,
    }[op](tenant=tenant, **params)

async def ensure_tenant(*, tenant: str, name: str | None = None, **_) -> dict:
    return await request(
        "POST", f"{settings().ticketing_url.rstrip('/')}/entities",
        token=settings().ticketing_token,
        json={"slug": tenant, "name": name or tenant},
    )

async def ensure_queue(*, tenant: str, queue: str, **_) -> dict:
    return await request(
        "POST", f"{settings().ticketing_url.rstrip('/')}/queues",
        token=settings().ticketing_token,
        json={"tenant": tenant, "name": queue},
    )

async def ensure_sla(*, tenant: str, matrix: dict | None = None, **_) -> dict:
    return await request(
        "POST", f"{settings().ticketing_url.rstrip('/')}/sla",
        token=settings().ticketing_token,
        json={"tenant": tenant, "matrix": matrix or {"P1": "4h", "P2": "8h", "P3": "NBD"}},
    )

async def create_work_order(
    *, tenant: str, title: str, queue: str = "saha", sla: str | None = None,
    acceptance: str | None = None, **_
) -> dict:
    """İnsan işi de kayıt altındadır: kabul kriteri olmadan iş emri kapanmaz."""
    return await request(
        "POST", f"{settings().ticketing_url.rstrip('/')}/tickets",
        token=settings().ticketing_token,
        json={
            "tenant": tenant,
            "title": title,
            "queue": queue,
            "sla": sla,
            "acceptance_criteria": acceptance,
            "requires_signoff": True,
        },
    )
```

platform/app/api/routers/agents.py

Ajan uçları; bağlantıyı her zaman ajan kurar

şerit: A – Aracı yazılım · 63 satır · python

```
"""Ajan uçları. Bağlantıyı her zaman ajan kurar; platform ajanı hiç aramaz."""
from datetime import datetime, timezone

from fastapi import APIRouter, Depends, HTTPException
from typing import Optional

from pydantic import BaseModel

from app.catalog import load_catalog
from app.config import settings
from app.security import agent_identity, issue_agent_token
from app.models import Lane
from app.services.spec_compiler import compile_spec

router = APIRouter(prefix="/agents", tags=["agents"])
_catalog = load_catalog(settings().catalog_dir)

class EnrollRequest(BaseModel):
    enrollment_code: str
    tenant: str
    device_uuid: str
    hostname: str
    serial: Optional[str] = None
    os_family: Optional[str] = None
    role: str = "workstation"

class ReportRequest(BaseModel):
    observed: dict[str, str] = {}
    inventory: dict = {}

@router.post("/enroll")
async def enroll(req: EnrollRequest) -> dict:
    if not settings().agent_enroll_secret:
        raise HTTPException(503, "kayıt sırrı tanımlı değil")
    token = issue_agent_token(tenant=req.tenant, device_uuid=req.device_uuid)
    return {"token": token, "checkin_seconds": settings().agent_checkin_seconds}

@router.post("/checkin")
async def checkin(req: ReportRequest, identity: tuple = Depends(agent_identity)) -> dict:
    """Ajan durumunu bildirir, karşılığında olması gereken ayarları alır."""
    tenant, device_uuid = identity
    spec = compile_spec(catalog=_catalog, tenant=tenant, entitlements=entitlements_for(tenant))
    todo = [t for t in spec.by_lane(Lane.A) if req.observed.get(f"{t.service_id}/{t.task_id}") != "ok"]
    return {
        "device": device_uuid,
        "seen_at": datetime.now(timezone.utc).isoformat(),
        "next_checkin": settings().agent_checkin_seconds,
        "tasks": [
            {"service_id": t.service_id, "task_id": t.task_id,
             "action": t.action, "params": t.params}
            for t in todo
        ],
    }

def _entitlements_for(tenant: str) -> list[str]:
    """Gerçek kurulumda veritabanından gelir; burada katalogdaki tümü."""
    return [s.id for s in _catalog]
```

platform/app/api/routers/specs.py

Tanım derleme, kuru çalıştırma ve dağıtım

şerit: D – Platform kurulumu · 60 satır · python

```
"""Tanım derleme, kuru çalıştırma ve dağıtım."""
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

from app.catalog import load_catalog
from app.config import settings
from app.services.dispatcher import dispatch
from app.services.spec_compiler import MissingSecret, compile_spec

router = APIRouter(prefix="/specs", tags=["specs"])
_catalog = load_catalog(settings().catalog_dir)

class CompileRequest(BaseModel):
    tenant: str
    entitlements: list[str]
    previous_services: list[str] = []
    device_roles: list[str] = []
    version: int = 1
    dry_run: bool = True

@router.post("/compile")
async def compile_endpoint(req: CompileRequest) -> dict:
    try:
        spec = compile_spec(
            catalog=_catalog,
            tenant=req.tenant,
            entitlements=req.entitlements,
            previous_services=req.previous_services,
            device_roles=set(req.device_roles),
            version=req.version,
        )
    except MissingSecret as exc:
        raise HTTPException(409, str(exc)) from None
    except Exception as exc:
        raise HTTPException(400, str(exc)) from None

    return {
        "spec": spec.to_json(),
        "summary": {
            "services": len(spec.services),
            "tasks": len(spec.tasks),
            "removed": spec.removed,
            "by_lane": {
                lane: len([t for t in spec.tasks if t.lane == lane]) for lane in "ABCDEF"
            },
        },
    }

@router.post("/apply")
async def apply_endpoint(req: CompileRequest) -> dict:
    spec = compile_spec(
        catalog=_catalog, tenant=req.tenant, entitlements=req.entitlements,
        previous_services=req.previous_services, device_roles=set(req.device_roles),
        version=req.version,
    )
    return {"results": await dispatch(spec, dry_run=req.dry_run)}
```

platform/app/catalog/services/switch-management.yaml

Örnek katalog kaydı — 39 hizmetin her biri böyle tanımlı

şerit: B — Uzaktan ayar · 47 satır · yaml

```
# Bu dosya tasarım belgesinden üretilmiştir; hizmet kimliği sitedeki kimlikle aynıdır.
id: switch-management
name: "Switch Management"
area: "Network Management"
tier: "Essential"
lane: B
lanes: [B, C, D, E]
plain: "Ofisteki bilgisayarları, yazıcıları ve telefonları birbirine bağlayan ağ anahtarlarının ayarlanması ve izlenmesi."
executor: "Site relay'den SSH (şifreli uzaktan bağlantı) ile Ansible cisco.ios/arista.eos/community.network"
automation_grade: ASSISTED
secret_stores: ["ansible-vault", "n8n-credentials"]
requires: [envanter-yonetim-sistemi]
health_check: "Zabbix SNMP erişilebilirliği, port bazlı LLD ifOperStatus, gecelik config-fark işi yeşil"
gotcha: "Hatalı trunk/VLAN basımı relay'in kendi yolunu keser; IOS'ta commit-confirm yoktur, play'leri 'reload in 5' ile sarın."
tasks:
  - id: vault-intake
    lane: D
    action: vault.store_access
    description: "Erişim bilgilerini kasaya yaz; parola koda gömülmez."
    params: {"store": ["ansible-vault", "n8n-credentials"]}
  - id: step-01
    lane: D
    action: netbox.ensure_device
    description: "NetBox API (uygulama arayüzü) ile cihazı, arayüzleri ve yönetim IP'sini (ağ adresi) oluştur"
  - id: step-02
    lane: B
    action: awx.launch
    description: "Baseline'ı bas: SNMPv3 (cihaz izleme protokolünün şifreli sürümü), syslog-relay, NTP (saat eşitleme),
AAA (kimlik doğrulama ve yetkilendirme), port-security, STP (ağ döngü koruması) koruyucuları"
  - id: step-03
    lane: D
    action: zabbix.ensure_host
    description: "Zabbix host'u + Cisco IOS (Cisco cihaz işletim sistemi) by SNMP (cihaz izleme protokolü) şablonu, arayüz
LLD'si (otomatik bileşen keşfi), uplink trigger'ları"
  - id: step-04
    lane: E
    action: n8n.schedule
    description: "n8n gecelik config çekimini git'e al, onaysız değişiklikte fark alarmı ver"
    params: {"workflow": "scheduled_generic", "cron": "0 6 1 * *"}
teardown:
  - id: revoke-access
    lane: D
    action: vault.revoke
    description: "Kasa anahtarlarını iptal et ve değiştir."
  - id: netbox-decommission
    lane: D
    action: netbox.decommission
    description: "Envanter kaydını hizmetten çıkarıldı olarak işaretle."
```

platform/tests/test_catalog.py

Tasarım kurallarını koda bağlayan testler

şerit: D – Platform kurulumu · 68 satır · python

```
"""Katalog bütünlüğü. Bu testler tasarımın kurallarını koda bağlar."""
import pytest

from app.catalog import load_catalog
from app.catalog.loader import CatalogError
from app.models import Lane

CATALOG_DIR = "app/catalog/services"

@pytest.fixture(scope="module")
def catalog():
    return load_catalog(CATALOG_DIR)

def test_yayindaki_hizmet_sayisi(catalog):
    assert len(catalog) == 39

def test_her_hizmetin_ilk_adimi_kasadir(catalog):
    """Kasaya yazılmamış bir yetkiyle hiçbir hizmet başlatılmaz."""
    for svc in catalog:
        assert svc.tasks[0].action == "vault.store_access", svc.id

def test_her_hizmetin_sokum_gorevi_var(catalog):
    """Çıkış hattı, giriş hattının aynadaki hâlidir."""
    for svc in catalog:
        assert svc.teardown, f"{svc.id}: söküm görevi yok"
        assert any(t.action == "vault.revoke" for t in svc.teardown), svc.id

def test_serit_dagilimi(catalog):
    counts = {lane: len(catalog.by_lane(lane)) for lane in Lane}
    assert counts[Lane.A] == 2      # yalnız Windows ve Linux Server Yönetimi
    assert counts[Lane.B] == 13
    assert counts[Lane.C] == 10
    assert counts[Lane.D] == 1
    assert counts[Lane.E] == 3
    assert counts[Lane.F] == 10
    assert sum(counts.values()) == 39

def test_envanter_her_seyin_on_kosulu(catalog):
    """Her hizmetin sağlık kontrolü envantere karşı çözülür."""
    envanter = catalog.get("envanter-yonetim-sistemi")
    assert envanter.requires == ()
    for svc in catalog:
        if svc.id != envanter.id:
            assert "envanter-yonetim-sistemi" in svc.requires, svc.id

def test_kasa_degerleri_gecerli(catalog):
    for svc in catalog:
        assert svc.secret_stores, svc.id
        for store in svc.secret_stores:
            assert store in {".env", "ansible-vault", "n8n-credentials"}

def test_bozuk_katalog_aciliste_patlar(tmp_path):
    (tmp_path / "kotu.yaml").write_text(
        "id: kotu\nname: Kötü\nlane: A\nautomation_grade: FULL\n"
        "secret_stores: ['.env']\ntasks:\n  - id: x\n    lane: A\n    action: agent.converge\n",
        encoding="utf-8",
    )
    with pytest.raises(CatalogError, match="vault.store_access"):
        load_catalog(tmp_path)
```

platform/tests/test_spec_compiler.py

Ön koşul, sökülme ve kasa kuralının testleri

şerit: D – Platform kurulumu · 64 satır · python

```
import pytest

from app.catalog import load_catalog
from app.models import Lane
from app.services.spec_compiler import MissingSecret, compile_spec
from app.services.task_graph import topological_order

CATALOG_DIR = "app/catalog/services"

@pytest.fixture(scope="module")
def catalog():
    return load_catalog(CATALOG_DIR)

def test_on_kosul_once_gelir(catalog):
    order = topological_order(catalog, ["switch-management"])
    ids = [s.id for s in order]
    assert ids[0] == "envanter-yonetim-sistemi"
    assert "switch-management" in ids

def test_hak_setinde_olmayan_on_kosul_da_kurulur(catalog):
    spec = compile_spec(catalog=catalog, tenant="acme", entitlements=["firewall-management"])
    assert "envanter-yonetim-sistemi" in spec.services

def test_paket_dusunce_sokum_gorevi_cikar(catalog):
    """Enterprise'dan Professional'a düşüş: kaldırılan hizmet sökülme görevi üretir."""
    spec = compile_spec(
        catalog=catalog,
        tenant="acme",
        entitlements=["envanter-yonetim-sistemi"],
        previous_services=["envanter-yonetim-sistemi", "immutable-backup"],
    )
    assert "immutable-backup" in spec.removed
    teardown = [t for t in spec.tasks if t.teardown]
    assert teardown and all(t.service_id == "immutable-backup" for t in teardown)

def test_ilk_gorev_her_zaman_kasa(catalog):
    spec = compile_spec(catalog=catalog, tenant="acme", entitlements=["active-directory"])
    assert spec.tasks[0].action == "vault.store_access"

def test_kasada_olmayan_sir_derlemeyi_durdurur(catalog):
    svc = catalog.get("switch-management")
    object.__setattr__(svc.tasks[1], "requires_secret", ["snmpv3"]) # noqa: SLF001
    with pytest.raises(MissingSecret):
        compile_spec(
            catalog=catalog, tenant="acme",
            entitlements=["switch-management"], vault_has=set(),
        )

def test_serit_dagilimi_gorevlerde_gorunur(catalog):
    spec = compile_spec(
        catalog=catalog, tenant="acme",
        entitlements=[s.id for s in catalog],
    )
    assert spec.by_lane(Lane.B), "B şeridinde görev yok"
    assert spec.by_lane(Lane.F), "F şeridinde görev yok"
    assert len(spec.services) == 39
```

.github/workflows/ci.yml

Şifresiz kasa, depoya girmiş .env ve açık metin sır kalıbını reddeder

şerit: D – Platform kurulumu · 72 satır · yaml

```
name: ci

on:
  push: {branches: [main]}
  pull_request: {}

jobs:
  secret-scan:
    name: Sır taraması
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Şifresiz ansible-vault dosyası var mı
        run: |
          fail=0
          while IFS= read -r f; do
            head -1 "$f" | grep -q '\${ANSIBLE_VAULT}' || { echo "ŞİFRESİZ: $f"; fail=1; }
          done < $(find . -path '*group_vars/*' -name 'vault*.yaml')
          exit $fail

      - name: Depoya girmiş .env dosyası var mı
        run: |
          found=$(git ls-files | grep -E '^(^/)\.env$' || true)
          [ -z "$found" ] || { echo "DEPODA .env: $found"; exit 1; }

      - name: Kaynak kodda açık metin sır kalıbı
        run: |
          if git ls-files -- '*.py' '*.yaml' '*.yml' '*.json' \
            | xargs grep -nEi "(password|secret|api[_-]?key|token)[\"]?s*[:=]\s*[\"]?[A-Za-z0-9/+=-._-]{12,}[\"]" \
            | grep -v -e example -e CHANGEEME -e '{\}' -e '\$env' -e lookup ; then
            echo "Açık metin sır bulundu"; exit 1
          fi

  platform:
    name: Platform testleri
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - working-directory: platform
        run: |
          pip install -e ".[dev]"
          python -m pytest -q
          ruff check .

  agent:
    name: Ajan testleri
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - working-directory: agent
        run: |
          pip install -e ".[dev]"
          python -m pytest -q
          ruff check .

  catalog:
    name: Katalog bütünlüğü
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - working-directory: platform
        run: |
```

```
pip install -e "[dev]"
python -m pytest tests/test_catalog.py -q
```

README.md

Beş bileşen, şerit karşılıkları ve kasa tablosu

şerit: D – Platform kurulumu · 60 satır · markdown

Hisar Delivery Platform

Hisar Research'ün yönetilen hizmet kataloğunu (yayında 39 hizmet) uçtan uca teslim eden otomasyon platformunun kaynak kodu.

> **Durum:** Bu depo, 'Hisar Hizmet Şeritleri' tasarım belgesinin koda dökülmüş hâlidir.
> Belgedeki altı yürütme şeridi (A-F) burada çalıştırılabilir bileşenlere karşılık gelir.

Beş bileşen

| # | Bileşen | Klasör | Ne yapar |
|---|------------------------------|-----------------|--|
| 1 | Orkestratör (n8n) | 'orchestrator/' | Sözleşme imzalanınca görev listesini derletir, işi şeritlere dağıtır, tekrar eden işleri takvimde koşturur |
| 2 | HR Platformu | 'platform/' | Beyin: katalog, hak seti, tanım derleyici, denetim kaydı, Zabbix/NetBox/SIEM/ticket entegrasyonları |
| 3 | Site Relay | 'relay/' | Müşteri sahasındaki aracı düğüm: iş kuyruğu, AWX yürütme düğümü, Zabbix proxy, paket önbelleği |
| 4 | İnsan | 'human/' | İş emri şablonları, yetki belgeleri, kabul kriterleri, runbook'lar |
| 5 | Aracı yazılım (agent) | 'agent/' | Uç cihaz ve sunucularda koşan Python ajanı: durum bildirir, tanımı çeker, uygular |

Şeritler ve kod karşılıkları

| Şerit | Ne yürütür | Kod |
|----------|--------------------------|---|
| A | Uç cihaz ajanı | 'agent/' |
| B | Ansible ile uzaktan ayar | 'relay/ansible/' + 'platform/app/integrations/awx.py' |
| C | Sağlayıcı arayüzleri | 'platform/app/integrations/' (netbox, zabbix, veeam, cloud) |
| D | HR platform kurulumu | 'platform/app/services/' + 'integrations/{zabbix,netbox,siem,ticketing}.py' |
| E | Takvimli işler | 'orchestrator/workflows/scheduled*.json' |
| F | İnsan iş emri | 'human/' + 'platform/app/integrations/ticketing.py' |

Sır yönetimi – istisnasız

Hiçbir parola, anahtar veya sertifika kaynak koda yazılmaz.

| Nerede çalışıyor | Kasa |
|------------------------------------|---|
| Agent ve makinede koşan script'ler | 'env' (bkz. 'agent/.env.example') |
| Ansible adımları | 'ansible-vault' (bkz. 'relay/ansible/group_vars/all/vault.yml') |
| n8n iş akışları ve API çağrıları | n8n credentials |
| Platform arka ucu | ortam değişkeni → 'platform/app/config.py' |

CI, 'env' ve vault dosyalarının şifresiz hâlde depoya girmesini engeller ('github/workflows/ci.yml').

Hızlı başlangıç

```
``bash
# Platform
cd platform && python -m venv .env && . .env/bin/activate
pip install -e "[dev]" && cp .env.example .env
uvicorn app.main:app --reload

# Agent (geliştirme)
cd agent && pip install -e "[dev]" && cp .env.example .env
python -m hragent --once

# Relay (müşteri sahası)
cd relay && cp .env.example .env && docker compose up -d
``
```

Ayrıntı: ['docs/architecture.md'](docs/architecture.md)

3 · Site Relay

Müşteri sahasındaki aracı düğüm: kuyruk, yürütme düğümü, izleme vekili

relay/docker-compose.yml

Relay yığını; hiçbirine internete açılmaz

şerit: A – Aracı yazılım B – Uzaktan ayar C – Sağlayıcı arayüzü · 57 satır · yaml

```
# Site Relay yığını. Hiçbiri gelen bağlantı için dış dünyaya açılmaz;
# yalnız müşterinin yerel ağına ve giden 443/27199'a ihtiyaç duyar.
services:
  redis:
    image: redis:7-alpine
    command: ["redis-server", "--requirepass", "${REDIS_PASSWORD}", "--appendonly", "yes"]
    volumes: ["redis-data:/data"]
    networks: [relay]
    restart: unless-stopped

  relay-api:
    build: ./relay_api
    env_file: [.env]
    environment:
      REDIS_URL: "redis://:${REDIS_PASSWORD}@redis:6379/0"
    ports:
      - "127.0.0.1:8080:8080" # yalnız yerel ağ; internete açılmaz
    depends_on: [redis]
    networks: [relay]
    restart: unless-stopped

  receptor:
    image: quay.io/ansible/receptor:latest
    volumes:
      - ./receptor/receptor.conf:/etc/receptor/receptor.conf:ro
      - receptor-data:/var/lib/receptor
    networks: [relay]
    restart: unless-stopped

  zabbix-proxy:
    image: zabbix/zabbix-proxy-sqlite3:alpine-7.0-latest
    environment:
      ZBX_SERVER_HOST: "${ZBX_SERVER_HOST}"
      ZBX_HOSTNAME: "${ZBX_HOSTNAME}"
      ZBX_PROXYMODE: "0" # etkin mod: proxy merkeze kendisi bağlanır
    ports:
      - "10051:10051" # yalnız müşteri yerel ağı – ajanlar buraya bildirir
    networks: [relay]
    restart: unless-stopped

  package-cache:
    image: nginx:alpine
    volumes:
      - ./package-cache/nginx.conf:/etc/nginx/nginx.conf:ro
      - package-data:/var/cache/packages
    ports: ["8081:80"] # yalnız müşteri yerel ağı
    networks: [relay]
    restart: unless-stopped

volumes:
  redis-data:
  receptor-data:
  package-data:

networks:
  relay:
```

relay/receptor/receptor.conf

Yürütme düğümü: gelen bağlantı dinlemez, kendisi dışarı bağlanır

şerit: B – Uzaktan ayar · 34 satır · ini

```
# AWX automation mesh – yürütme düğümü.
#
# Kritik iki ayar:
# * listener yok          → düğüm gelen bağlantı DİNLEMEZ
# * tcp-peer (giden)      → bağlantıyı düğüm kendisi kurar
# AWX tarafında bu düğüm `listener_port: null` ve `peers_from_control_nodes: false`
# ile kaydedilir. Böylece müşteri güvenlik duvarında gelen kural açılmaz.

- node:
  id: relay-{{RELAY_TENANT}}

- log-level: info

- tcp-peer:
  address: {{RECEPTOR_PEER}}
  redial: true
  tls: mesh-client

- tls-client:
  name: mesh-client
  rootcas: /var/lib/receptor/tls/ca.crt
  cert: /var/lib/receptor/tls/relay.crt
  key: /var/lib/receptor/tls/relay.key

- control-service:
  service: control
  filename: /var/run/receptor/receptor.sock

- work-command:
  worktype: ansible-runner
  command: ansible-runner
  params: worker
  allowruntimeparams: true
```

relay/relay_api/main.py

Ajanların tek muhatabı

şerit: A – Aracı yazılım · 61 satır · python

```
"""Relay API – ajanların tek muhatabı.

Ajan platformu doğrudan görmez; her şey relay üzerinden geçer. Bu, müşteri açısından
dışarı tek bir çıkış noktası olmasını sağlar.
"""
from __future__ import annotations

import os

import structlog
from fastapi import FastAPI, Header, HTTPException
from pydantic import BaseModel

import queue as jobqueue

log = structlog.get_logger(__name__)
app = FastAPI(title="Hisar Site Relay", version="0.1.0")

TENANT = os.environ.get("RELAY_TENANT", "")
RELAY_TOKEN = os.environ.get("RELAY_TOKEN", "")


class Task(BaseModel):
    service_id: str
    task_id: str
    action: str
    params: dict = {}
    device_uuid: str | None = None
    teardown: bool = False


class Checkin(BaseModel):
    device_uuid: str
    observed: dict = {}
    inventory: dict = {}


def _require_platform(authorization: str) -> None:
    if not RELAY_TOKEN or authorization != f"Bearer {RELAY_TOKEN}":
        raise HTTPException(401, "yetkisiz")


@app.get("/healthz")
async def healthz() -> dict:
    return {"status": "ok", "tenant": TENANT, "queue_depth": await jobqueue.depth(TENANT)}


@app.post("/queue")
async def enqueue(task: Task, authorization: str = Header(default="")) -> dict:
    """Platform görev bırakır."""
    _require_platform(authorization)
    return await jobqueue.push(TENANT, task.model_dump())


@app.post("/agents/checkin")
async def checkin(req: Checkin) -> dict:
    """Ajan yoklaması: durumunu bildirir, bekleyen görevlerini alır."""
    tasks = await jobqueue.pop_all(TENANT, req.device_uuid)
    log.info("ajan_yoklamasi", cihaz=req.device_uuid, gorev=len(tasks))
    return {"tasks": tasks, "next_checkin": int(os.environ.get("CHECKIN_SECONDS", "45"))}
```

relay/relay_api/queue.py

İş kuyruğu; cihaz kimliğine göre bölümlenir

şerit: A – Aracı yazılım · 67 satır · python

```
"""Relay iş kuyruğu.

Platform görevi buraya bırakır; ajan bir sonraki yoklamasında çeker. Ajan hiçbir zaman
aranmaz – yön her zaman içeriden dışarıdır. Kuyruk cihaz kimliğine göre bölümlenir ki
bir cihazın işi başka cihaza gitmesin.
"""

from __future__ import annotations

import json
import os

import redis.asyncio as redis
import structlog

log = structlog.get_logger(__name__)

TTL_SECONDS = 24 * 3600
_client: redis.Redis | None = None

def client() -> redis.Redis:
    global _client
    if _client is None:
        _client = redis.from_url(
            os.environ.get("REDIS_URL", "redis://localhost:6379/0"), decode_responses=True
        )
    return _client

def _key(tenant: str, device_uuid: str) -> str:
    return f"queue:{tenant}:{device_uuid or 'unassigned'}"

async def push(tenant: str, task: dict) -> dict:
    key = _key(tenant, task.get("device_uuid", ""))
    r = client()
    await r.rpush(key, json.dumps(task, ensure_ascii=False))
    await r.expire(key, TTL_SECONDS)
    depth = await r.llen(key)
    log.info("gorev_kuyruğa_alindi", kuyruk=key, derinlik=depth)
    return {"queued": True, "queue": key, "depth": depth}

async def pop_all(tenant: str, device_uuid: str, limit: int = 50) -> list[dict]:
    """Ajan yoklamasında bekleyen görevleri alır. Alınan görev kuyruktan düşer."""
    key = _key(tenant, device_uuid)
    r = client()
    out: list[dict] = []
    for _ in range(limit):
        raw = await r.lpop(key)
        if raw is None:
            break
        out.append(json.loads(raw))
    # cihaza özel kuyruk boşsa, atanmamış havuzdan da al
    if not out and device_uuid:
        shared = _key(tenant, "")
        for _ in range(limit):
            raw = await r.lpop(shared)
            if raw is None:
                break
            out.append(json.loads(raw))
    return out

async def depth(tenant: str, device_uuid: str = "") -> int:
    return int(await client().llen(_key(tenant, device_uuid)))
```

relay/ansible/ansible.cfg

Parolalar playbook'ta değil kasada

şerit: B – Uzaktan ayar · 17 satır · ini

```
[defaults]
inventory = inventory/netbox.yml
roles_path = roles
host_key_checking = True
stdout_callback = yaml
# Parolalar playbook'un içinde durmaz; ansible-vault'tan çözülür.
vault_password_file = /var/lib/receptor/vault-pass
retry_files_enabled = False

[privilege_escalation]
become = True
become_method = sudo

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=yes
```

relay/ansible/inventory/netbox.yml

Makine listesi elle tutulmaz; envanterden gelir

şerit: B – Uzaktan ayar · 17 satır · yaml

```
# Envanter doğruluk kaynağıdır: makine listesi elle tutulmaz, NetBox'tan gelir.
# Böylece bir cihaz envantere girmeden hiçbir playbook onu göremez.
plugin: netbox.netbox.nb_inventory
api_endpoint: "{{ lookup('env', 'NETBOX_URL') }}"
token: "{{ lookup('env', 'NETBOX_TOKEN') }}" # değer kasadan gelir
validate_certs: true
config_context: false
group_by:
  - device_roles
  - tenants
  - sites
query_filters:
  - status: active
  - tenant: "{{ lookup('env', 'RELAY_TENANT') }}" # yalnız bu müşterinin cihazları
compose:
  ansible_host: primary_ip4.address | default(name)
```

relay/ansible/roles/switch_baseline/tasks/main.yml

Geri sayımlı yeniden başlatma ile korunan switch ayarı

şerit: B – Uzaktan ayar · 37 satır · yaml

```
---
# Switch temel ayarı. Hatalı bir basım relay'in kendi yolunu keser; IOS'ta
# commit-confirm yoktur, bu yüzden geri sayımlı yeniden başlatma ile korunuruz.
- name: Geri alma ağı kur (5 dakika içinde onaylanmazsa cihaz eski config ile açılır)
  cisco.ios.ios_command:
    commands:
      - command: "reload in 5"
        prompt: "Proceed with reload"
        answer: "y"
    changed_when: false

- name: Çalışan yapılandırmayı yedekle
  cisco.ios.ios_config:
    backup: true
    backup_options:
      dir_path: /var/lib/receptor/config-backups
      filename: "{{ inventory_hostname }}-{{ ansible_date_time.iso8601_basic_short }}.cfg"

- name: Temel ayarları uygula
  cisco.ios.ios_config:
    lines:
      - "ntp server {{ ntp_servers[0] }}"
      - "logging host {{ syslog_target }}"
      - "snmp-server group hr v3 priv"
      - "spanning-tree portfast bpduguard default"
      - "spanning-tree loopguard default"
    notify: onayla ve geri alma iptal

- name: SNMPv3 kullanıcısı (parola kasadan gelir)
  cisco.ios.ios_config:
    lines:
      - >-
        snmp-server user {{ snmpv3_user }} hr v3 auth sha {{ snmpv3_auth_pass }}
        priv aes 128 {{ snmpv3_priv_pass }}
    no_log: true # parola hiçbir günlüğe düşmez
    notify: onayla ve geri alma iptal
```

relay/ansible/roles/firewall_baseline/tasks/main.yml

HA sanal adresine değil birincil düğüme basar

şerit: B – Uzaktan ayar · 35 satır · yaml

```
---
# Güvenlik duvarı temel ayarı. Yapılandırma HA sanal adresine değil, birincil düğümün
# gerçek yönetim adresine basılır; aksi hâlde devir anında fark kaybolabilir.
- name: Birincil düğümü doğrula
  fortinet.fortios.fortios_monitor_fact:
    selector: system_ha-peer
  register: ha
  failed_when: ha.meta.results is undefined

- name: Yapılandırmayı yedekle
  fortinet.fortios.fortios_monitor:
    selector: system.config.backup
    params: {scope: global}

- name: Yönetim erişimini yalnız relay'e sınırla
  fortinet.fortios.fortios_system_interface:
    system_interface:
      name: "{{ mgmt_interface | default('mgmt') }}"
      allowaccess: "ping https ssh"
      trusthost1: "{{ relay_cidr }}"

- name: Yönetici çok adımlı doğrulaması
  fortinet.fortios.fortios_system_admin:
    system_admin:
      name: "{{ hr_admin_user }}"
      two_factor: "fortitoken"
  no_log: true

- name: Kayıtları relay toplayıcısına gönder
  fortinet.fortios.fortios_log_syslogd_setting:
    log_syslogd_setting:
      status: "enable"
      server: "{{ syslog_target }}"
      mode: "reliable"
```

relay/ansible/playbooks/site.yml

Hangi rolün koşacağını envanterdeki cihaz rolü belirler

şerit: B – Uzaktan ayar · 20 satır · yaml

```
---
# Ana giriş noktası. AWX iş şablonu bunu çağırır; hangi rolün koşacağını
# NetBox'taki cihaz rolü belirler – elle host listesi tutulmaz.
- name: Ağ anahtarları
  hosts: device_roles_switch
  gather_facts: false
  connection: network_cli
  roles: [switch_baseline]

- name: Güvenlik duvarları
  hosts: device_roles_firewall
  gather_facts: false
  connection: httpapi
  roles: [firewall_baseline]

- name: Etki alanı denetleyicileri
  hosts: device_roles_domain_controller
  gather_facts: true
  roles: [ad_baseline]
```

relay/ansible/playbooks/teardown.yml

Çıkış hattı: giriş hattının aynadaki hâli

şerit: B – Uzaktan ayar D – Platform kurulumu · 21 satır · yaml

```
---
# Çıkış hattı: giriş hattının aynadaki hâli. Sözleşme bitince koşar.
- name: Erişimleri geri al
  hosts: all
  gather_facts: false
  tasks:
    - name: HR yönetim erişimini kaldır
      ansible.builtin.include_role:
        name: "{{ device_role }}_baseline"
      tasks_from: teardown
      when: device_role is defined

    - name: Kasa anahtarını iptal et
      ansible.builtin.uri:
        url: "{{ lookup('env', 'VAULT_ADDR') }}/v1/secret/metadata/tenants/{{ tenant }}/{{ inventory_hostname }}"
        method: DELETE
        headers:
          X-Vault-Token: "{{ lookup('env', 'VAULT_TOKEN') }}"
      delegate_to: localhost
      no_log: true
```

relay/ansible/group_vars/all/vault.yml

ansible-vault ile şifreli; şifresiz hâli CI tarafından reddedilir

şerit: B – Uzaktan ayar · 15 satır · yaml

```
$ANSIBLE_VAULT;1.1;AES256
# Bu dosya ansible-vault ile şifrelenmiştir. Depoda yalnız şifreli hâli bulunur.
# Şifresiz hâlde depoya girmesini CI engeller (.github/workflows/ci.yml → secret-scan).
#
# İçeriği (çözülünce):
#   vault_switch_enable_password: ...
#   vault_snmpv3_auth_pass: ...
#   vault_snmpv3_priv_pass: ...
#   vault_firewall_api_key: ...
#   vault_ad_service_account_password: ...
#   vault_veeam_api_token: ...
#
# ÖRNEK yer tutucu – gerçek dosya `ansible-vault encrypt` çıktısıdır.
36613464656661383661383964363638393764373862623565383537383363396533393264653938
```

4 · İnsan

İş emri, yetki belgesi ve kabul formu şablonları

human/templates/access_pack.md

0. gün erişim paketi; liste paketten otomatik üretilir

şerit: F — İnsan işi · 26 satır · markdown

```
# 0. Gün Erişim Paketi — {{ müşteri }}

Bu liste otomatik üretilmiştir: yalnız `{{ paket }}` paketinin gerektirdiği erişimler
istenir. Gereksiz hiçbir yetki talep edilmez, gereken hiçbir yetki unutulmaz.

Her kalem alındığı anda kendi kasasına yazılır ve bir daha açık metin olarak durmaz.

#	Erişim	Nereye yazılır	Kaç hizmet buna bağlı	Teslim alındı
{% for item in erisimler %}| {{ loop.index }} | {{ item.ad }} | `{{ item.kasa }}` | {{ item.bagimli_hizmet }} | □ |
{% endfor %}

## Saha erişimi

- [ ] Lokasyon listesi ve adresler
- [ ] Bina giriş izni / kart veya refakat düzeni
- [ ] Kabinet anahtarı
- [ ] Site relay için rack veya sanal makine kapasitesi, yönetim ağı ve giden 443 izni

## Kapanış

Paket tamamlanmadan hiçbir hizmet başlatılmaz. Her kalem kasaya yazıldıktan sonra
otomatik kanarya testi koşar; yeşil dönmeden hizmet başlatılmış sayılmaz.**

İmza (müşteri yetkilisi): _____ Tarih: _____
```

human/templates/acceptance.md

Proje → yönetilen devri; sağlık yeşil olmadan kapanmaz

şerit: F — İnsan işi · 21 satır · markdown

```
# Kabul Formu — Proje'den Yönetilen Hizmete Devir

Kurulum işi, devralacak hizmetin ilk sağlık kontrolü yeşile dönmeden kapanmaz.
Bu form, "kuruldu ama izlenmiyor" ve "teslim edildi ama yedeği yok" boşluğunu kapatır.

Alan	Değer
Proje hizmeti	{{ proje_hizmeti }}
Devralan yönetilen hizmet	{{ yönetilen_hizmet }}
Devir tarihi	{{ tarih }}

## Kapanış koşulları

- [ ] Envanter kaydı oluşturuldu ve doğrulandı
- [ ] İzleme host'u açıldı, şablonu bağlandı, ilk veri geldi
- [ ] Yedekleme kapsamına alındı ve ilk geri dönüş noktası oluştu
- [ ] Erişim bilgileri kasaya yazıldı, kanarya testi yeşil
- [ ] Sağlık kontrolü yeşil — ekran görüntüsü eklendi

Saha ekibi: _____ Devralan: _____ Müşteri: _____
```

human/templates/pentest_authorization.md

İmzasız test başlamaz

şerit: F – İnsan işi · 20 satır · markdown

```
# Sızma Testi Yetki Mektubu

> **Bu mektup imzalanmadan test başlamaz.** Yetkisiz test suç teşkil eder; kapsam dışı
> bir sistem etkilenirse sorumluluk hizmet sağlayıcıya geçer.

Alan	Değer
Müşteri	{{ müşteri }}
Test penceresi	{{ baslangic }} - {{ bitis }}
Kapsam (adres / alan adı)	{{ kapsam }}
**Kapsam dışı**	{{ kapsam_disi }}
Acil durdurma iletişimi	{{ acil_irtibat }}

## Kurallar
- Hizmet kesintisine yol açabilecek testler yalnız yazılı ek onayla yapılır.
- Endüstriyel kontrol cihazları ve eski yazıcılar varsayılan olarak kapsam dışıdır.
- Elde edilen veriler test bitiminde imha edilir; imha kaydı rapora eklenir.

Müşteri yetkilisi: _____ Unvan: _____ Tarih: _____ İmza: _____
```

human/runbooks/dr_failover.md

Failover'ı kimin ilan edeceği isimle yazılı

şerit: F – İnsan işi · 22 satır · markdown

```
# Felaket Kurtarma – Devir Runbook'u

**Failover'ı ilan etmeye yetkili kişi burada isimle yazılıdır.** Yetki nesnesi
tanımlı değilse otomasyon bekler; bu bilinçli bir tasarım kararıdır.

Rol	İsim	İletişim
Failover ilan yetkisi	{{ yetkili }}	{{ iletisim }}
Müşteri onayı	{{ müşteri_yetkili }}	{{ müşteri_iletisim }}

## Sıra
1. Olayı doğrula – izleme ve saha teyidi
2. Yetkili failover ilan eder (sözlü yeterli değil, kayıt açılır)
3. Adres değişikliği ve alan adı süreleri: kesimden **önce** düşürülmüş olmalı
4. Replikaları etkinleştir, açılış sırasını envanterden oku
5. Güvenlik duvarı kurallarını uygulanmamış şablondan aç
6. Uygulama seviyesinde kabul testi
7. Müşteriye bildir, kaydı kapat

> Süreyi replikasyon değil adres değişikliği, alan adı süresi ve lisans yeniden
> aktivasyonu belirler. Tatbikatta bunlar ölçülür.
```

5 · Uç Nokta Ajanı

Uç cihazda koşan Python ajanı

agent/hragent/__main__.py

Ajan döngüsü; ilk gün ile 400. gün aynı

şerit: A – Aracı yazılım · 73 satır · python

```
"""Ajan döngüsü.

Devreye alma ile günlük işletme aynı döngüdür: ilk gün her şey eksiktir,
sonraki turlarda yalnız sapanlar uygulanır.
"""

from __future__ import annotations

import argparse
import sys
import time

import structlog

from hragent import inventory as inv
from hragent.client import Client
from hragent.config import Config
from hragent.runner import UnknownAction, run_task

log = structlog.get_logger(__name__)

def one_cycle(client: Client, observed: dict, *, dry_run: bool = False) -> dict:
    facts = inv.collect()
    response = client.checkin(observed, facts)
    tasks = response.get("tasks", [])
    log.info("gorev_alindi", sayi=len(tasks), cihaz=facts["uuid"])

    for task in tasks:
        try:
            result = run_task(task, dry_run=dry_run)
            observed[result["key"]] = result["state"]
        except UnknownAction as exc:
            log.warning("bilinmeyen_eylem", error=str(exc))
        except Exception as exc: # noqa: BLE001 – tek görev hatası döngüyü düşürmez
            key = f"{task.get('service_id')}/{task.get('task_id')}"
            observed[key] = "failed"
            log.error("gorev_hatasi", gorev=key, error=str(exc))

    return response

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(prog="hragent")
    parser.add_argument("--once", action="store_true", help="tek tur koş ve çık")
    parser.add_argument("--dry-run", action="store_true", help="uygulama, yalnız raporla")
    args = parser.parse_args(argv)

    config = Config.load()
    if not config.platform_url:
        log.error("yapilandirma_eksik", eksik="HRAGENT_PLATFORM_URL")
        return 2

    client = Client(config)
    if not config.token and config.enrollment_code:
        client.config.token = client.enroll(inv.collect())
        log.info("kayit_tamam")

    observed: dict = {}
    while True:
        try:
            response = one_cycle(client, observed, dry_run=args.dry_run)
            wait = int(response.get("next_checkin", config.checkin_seconds))
        except Exception as exc: # noqa: BLE001 – ağ kesintisinde susmayıp beklemeye devam
            log.error("tur_basarisiz", error=str(exc))
            wait = config.checkin_seconds

        if args.once:
            return 0

        time.sleep(wait)
```

```
if __name__ == "__main__":
    sys.exit(main())
```

agent/hragent/client.py

Bağlantıyı her zaman ajan kurar: giden 443

şerit: A – Aracı yazılım · 50 satır · python

```
"""Platform istemcisi.

Bağlantıyı her zaman ajan kurar: giden 443. Müşterinin güvenlik duvarında
HR için gelen kural açılmaz.
"""
from __future__ import annotations

import httpx
import structlog

from hragent.config import Config

log = structlog.get_logger(__name__)

class Client:
    def __init__(self, config: Config) -> None:
        self.config = config

    def _headers(self) -> dict:
        return {"Authorization": f"Bearer {self.config.token}"} if self.config.token else {}

    def enroll(self, inventory: dict) -> str:
        """Kayıt: cihaza özel belirteç alınır. Kayıt kodu tek kullanımlıktır."""
        with httpx.Client(timeout=30) as client:
            resp = client.post(
                f"{self.config.platform_url}/agents/enroll",
                json={
                    "enrollment_code": self.config.enrollment_code,
                    "tenant": self.config.tenant,
                    "device_uuid": inventory["uuid"],
                    "hostname": inventory["hostname"],
                    "serial": inventory.get("serial"),
                    "os_family": inventory.get("os_family"),
                },
            )
            resp.raise_for_status()
            return resp.json()["token"]

    def checkin(self, observed: dict, inventory: dict) -> dict:
        """Durum bildirilir, karşılığında yapılacak görevler alınır."""
        with httpx.Client(timeout=30) as client:
            resp = client.post(
                f"{self.config.platform_url}/agents/checkin",
                headers=self._headers(),
                json={"observed": observed, "inventory": inventory},
            )
            resp.raise_for_status()
            return resp.json()
```

agent/hragent/runner.py

Yalnız A şeridini yürütür; Ansible işini reddeder

şerit: A – Aracı yazılım · 41 satır · python

```
"""Görev yürütücü: platformdan gelen eylemi uygun uygulayıcıya bağlar."""
from __future__ import annotations

import structlog

from hragent.apply import ensure_file, ensure_package, ensure_service

log = structlog.get_logger(__name__)

class UnknownAction(Exception):
    pass

def run_task(task: dict, *, dry_run: bool = False) -> dict:
    """Tek bir görevi uygular ve sonucu döndürür. İstisna yutulmaz, yukarı taşınır."""
    action = task.get("action", "")
    params = task.get("params", {}) or {}
    key = f"{task.get('service_id')}/{task.get('task_id')}"

    if action != "agent.converge":
        raise UnknownAction(f"{key}: ajan bu eylemi yürütmez → {action}")

    results = []
    for pkg in params.get("packages", []):
        results.append(ensure_package(pkg["name"], version=pkg.get("version"), dry_run=dry_run))
    for svc in params.get("services", []):
        results.append(ensure_service(svc["name"], state=svc.get("state", "running"),
                                     dry_run=dry_run))
    for f in params.get("files", []):
        results.append(ensure_file(f["path"], f["content"], mode=int(f.get("mode", "644"), 8),
                                   secret=f.get("secret", False), dry_run=dry_run))

    failed = [r for r in results if r["state"] == "failed"]
    return {
        "key": key,
        "state": "failed" if failed else "ok",
        "changed": any(r["changed"] for r in results),
        "details": results,
    }
```

agent/hragent/secrets.py

`.env` okur; fazla açık izinli dosyayı reddeder

şerit: A – Aracı yazılım · 60 satır · python

```
"""Sır okuma.

Ajanın çalıştırdığı hiçbir script parolayı içinde taşımaz. Değerler `.env` dosyasından
okunur; dosya 0600 izinleriyle durur ve depoya asla girmez.
"""

from __future__ import annotations

import os
import stat
from pathlib import Path

import structlog

log = structlog.get_logger(__name__)

DEFAULT_PATHS = (
    Path("/etc/hragent/.env"),
    Path("C:/ProgramData/HRAgent/.env"),
    Path(".env"),
)

class InsecureSecretFile(Exception):
    """Sır dosyası fazla açık izinlerle duruyorsa okumayı reddederiz."""

def _check_permissions(path: Path) -> None:
    if os.name == "nt": # Windows'ta izin modeli farklı, ACL ayrıca kontrol edilir
        return
    mode = path.stat().st_mode
    if mode & (stat.S_IRGRP | stat.S_IROTH):
        raise InsecureSecretFile(f"{path}: grup/diğer okuyabiliyor, 0600 olmalı")

def load(path: str | None = None) -> dict:
    """`.env` dosyasını okur. Değerler süreç dışına hiç çıkmaz."""
    candidates = [Path(path)] if path else list(DEFAULT_PATHS)
    for candidate in candidates:
        if not candidate.is_file():
            continue
        _check_permissions(candidate)
        values = {}
        for line in candidate.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, _, value = line.partition("=")
            values[key.strip()] = value.strip().strip('"').strip("'")
        log.info("sirlar_yuklendi", kaynak=str(candidate), anahtar_sayisi=len(values))
        return values
    log.warning("sir_dosyasi_bulunamadi", aranan=[str(c) for c in candidates])
    return {}

def get(key: str, default: str = "") -> str:
    """Önce ortam değişkeni, sonra `.env`. Koda gömülü varsayılan parola yoktur."""
    if key in os.environ:
        return os.environ[key]
    return load().get(key, default)
```

agent/hragent/inventory.py

Kimlik anahtarı kalıcı ajan kimliği — klon makineler için

şerit: A — Aracı yazılım · 79 satır · python

```
"""Donanım ve yazılım envanteri.

Kimlik anahtarı hostname veya ağ kartı kimliği DEĞİLDİR: klonlanmış sanal makineler
ikisini de paylaşır ve iki varlık tek kayda çöker. Anahtar, kalıcı ajan kimliği +
seri numarasıdır.
"""

from __future__ import annotations

import platform
import subprocess
import uuid
from pathlib import Path

import structlog

log = structlog.get_logger(__name__)

UUID_FILE = Path("/etc/hragent/uuid") if platform.system() != "Windows" \
    else Path("C:/ProgramData/HRAgent/uuid")

def device_uuid() -> str:
    """Kalıcı ajan kimliği. Bir kez üretilir, diskte kalır."""
    if UUID_FILE.is_file():
        value = UUID_FILE.read_text(encoding="utf-8").strip()
        if value:
            return value
    value = str(uuid.uuid4())
    try:
        UUID_FILE.parent.mkdir(parents=True, exist_ok=True)
        UUID_FILE.write_text(value, encoding="utf-8")
    except OSError as exc:
        log.warning("kimlik_yazilamadi", error=str(exc))
    return value

def serial_number() -> str:
    """Üretici seri numarası. Boş veya yinelenen olabilir — tek başına anahtar değildir."""
    system = platform.system()
    try:
        if system == "Linux":
            path = Path("/sys/class/dmi/id/product_serial")
            return path.read_text(encoding="utf-8").strip() if path.is_file() else ""
        if system == "Darwin":
            # ioreg çıktısı geçerli UTF-8 olmayabilir: bayt olarak alıp hoşgörülü çözeriz,
            # ayrıca tüm ağaç yerine yalnız platform düğümünü sorarız.
            raw = subprocess.run(
                ["ioreg", "-c", "IOPlatformExpertDevice", "-d", "2"],
                capture_output=True, timeout=10, check=False,
            ).stdout
            out = raw.decode("utf-8", errors="replace")
            for line in out.splitlines():
                if "IOPlatformSerialNumber" in line:
                    parts = line.split(' ')
                    return parts[-2] if len(parts) >= 2 else ""
            return ""
        if system == "Windows":
            raw = subprocess.run(
                ["wmic", "bios", "get", "serialnumber"],
                capture_output=True, timeout=10, check=False,
            ).stdout
            out = raw.decode("utf-8", errors="replace").splitlines()
            return out[1].strip() if len(out) > 1 else ""
    except (OSError, subprocess.SubprocessError, IndexError, UnicodeDecodeError) as exc:
        log.warning("seri_no_okunamadi", error=str(exc))
    return ""

def collect() -> dict:
    return {
        "uuid": device_uuid(),
```

```

    "serial": serial_number(),
    "hostname": platform.node(),
    "os_family": platform.system(),
    "os_release": platform.release(),
    "arch": platform.machine(),
    "python": platform.python_version(),
}

```

agent/hragent/apply/packages.py

Paketler relay önbelleğinden; kurulu ise dokunmaz

şerit: A – Aracı yazılım · 55 satır · python

```

"""Paket kurulumu. Paketler site relay'in önbelleğinden çekilir – geniş alan ağı doymasın."""
from __future__ import annotations

import platform
import subprocess

import structlog

log = structlog.get_logger(__name__)

MANAGERS = {
    "Windows": ["winget", "install", "--silent", "--accept-package-agreements", "--id"],
    "Linux": ["apt-get", "install", "-y"],
    "Darwin": ["brew", "install"],
}

class UnsupportedPlatform(Exception):
    pass

def _installed(name: str) -> bool:
    system = platform.system()
    probe = {
        "Windows": ["winget", "list", "--id", name],
        "Linux": ["dpkg", "-s", name],
        "Darwin": ["brew", "list", name],
    }.get(system)
    if not probe:
        raise UnsupportedPlatform(system)
    return subprocess.run(probe, capture_output=True, check=False).returncode == 0

def ensure_package(name: str, *, version: str | None = None, dry_run: bool = False) -> dict:
    """İstenen paketin kurulu olmasını sağlar. Kurulu ise hiçbir şey yapmaz."""
    if _installed(name):
        return {"action": "package", "name": name, "state": "ok", "changed": False}
    if dry_run:
        return {"action": "package", "name": name, "state": "would-install", "changed": False}

    system = platform.system()
    cmd = MANAGERS.get(system)
    if not cmd:
        raise UnsupportedPlatform(system)
    target = f"{name}={version}" if version and system == "Linux" else name
    result = subprocess.run([*cmd, target], capture_output=True, text=True, check=False)
    ok = result.returncode == 0
    if not ok:
        log.error("paket_kurulamadi", paket=name, kod=result.returncode, hata=result.stderr[:400])
    return {
        "action": "package", "name": name,
        "state": "ok" if ok else "failed",
        "changed": ok, "exit_code": result.returncode,
    }

```

agent/hragent/apply/files.py

Sır dosyaları daima 0600

şerit: A — Aracı yazılım · 37 satır · python

```
"""Dosya içeriği ve izinleri. Sır dosyaları daima 0600 yazılır."""
from __future__ import annotations

import hashlib
import os
from pathlib import Path

SECRET_MODE = 0o600

def _digest(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def ensure_file(
    path: str, content: str, *, mode: int = 0o644, secret: bool = False, dry_run: bool = False
) -> dict:
    target = Path(path)
    desired = content.encode("utf-8")
    effective_mode = SECRET_MODE if secret else mode

    if target.is_file() and _digest(target.read_bytes()) == _digest(desired):
        if os.name != "nt" and (target.stat().st_mode & 0o777) != effective_mode:
            if not dry_run:
                target.chmod(effective_mode)
            return {"action": "file", "path": path, "state": "ok", "changed": True}
        return {"action": "file", "path": path, "state": "ok", "changed": False}

    if dry_run:
        return {"action": "file", "path": path, "state": "would-write", "changed": False}

    target.parent.mkdir(parents=True, exist_ok=True)
    target.write_bytes(desired)
    if os.name != "nt":
        target.chmod(effective_mode)
    return {"action": "file", "path": path, "state": "ok", "changed": True}
```

agent/tests/test_runner.py

Ajanın kendi şeridi dışına çıkmadığının testi

şerit: A — Aracı yazılım · 29 satır · python

```
import pytest

from hragent.runner import UnknownAction, run_task

def test_ajan_kendi_seridinde_olmayan_eylemi_reddeder():
    """Ajan yalnız A şeridini yürütür; Ansible işi ona verilmez."""
    with pytest.raises(UnknownAction):
        run_task({"service_id": "switch-management", "task_id": "step-02", "action": "awx.launch"})

def test_bos_gorev_basarili_ve_degisiklik_yok():
    result = run_task(
        {"service_id": "x", "task_id": "t", "action": "agent.converge", "params": {}}
    )
    assert result["state"] == "ok" and result["changed"] is False

def test_dosya_gorevi_uygulanir(tmp_path):
    target = tmp_path / "zabbix_agent2.conf"
    result = run_task(
        {
            "service_id": "server-monitoring", "task_id": "step-01", "action": "agent.converge",
            "params": {"files": [{"path": str(target), "content": "ServerActive=relay\n"}]},
        }
    )
    assert result["state"] == "ok" and result["changed"] is True
    assert target.read_text(encoding="utf-8") == "ServerActive=relay\n"
```

Şerit kısaltmaları: A — aracı yazılım · B — uzaktan ayar · C — sağlayıcı arayüzü · D — platform kurulumu · E — takvimli iş · F — insan işi.